

APPUNTI DI

ELETTRONICA DIGITALE

GIOSUÈ AIELLO

Indice

1	Segnali, algebra di Boole e reti logiche	4
1.1	Segnali analogici e digitali	4
1.2	Algebra di Boole	4
1.2.1	Il concetto di duale	4
1.3	Cenni storici	5
1.4	Reti logiche	5
1.4.1	Funzioni a 1 ingresso	5
1.4.2	Funzioni a 2 ingressi	6
1.4.3	Funzioni a n ingressi	6
1.5	Bit di parità	6
1.6	Esercizio: semplice circuito per abilitare la presa dati	7
2	Realizzazione fisica delle porte logiche	8
2.1	Esercizio: bit di parità a 4 bit	8
2.2	Verifica di una trasmissione tramite parità	9
2.3	Caratteristiche fisiche delle porte logiche	9
2.4	Implementazione delle porte logiche	9
2.4.1	NOT con un singolo transistor	9
2.4.2	NOT in tecnologia TTL	10
2.5	Immunità al rumore	10
2.6	Nomenclatura delle famiglie logiche e fan-out	11
2.6.1	Nomenclatura dei componenti	11
2.6.2	Fan-Out (capacità di pilotaggio)	11
3	Algebra di Boole: assiomi, proprietà e sintesi	13
3.1	Tempi caratteristici delle porte logiche	13
3.2	Assiomi dell'algebra di Boole	13
3.3	Proprietà dell'algebra di Boole	13
3.3.1	Teorema della semplificazione	13
3.3.2	Teorema della moltiplicazione e fattorizzazione	13
3.3.3	Leggi di De Morgan	14
3.4	Porte universali: NAND e NOR	14
3.4.1	NAND come porta universale	14
3.4.2	NOR come porta universale	14
3.5	Dalla tabella di verità al circuito	14
3.5.1	Equivalenza delle due forme canoniche	15
3.5.2	I circuiti che corrispondono alle due forme	15
3.5.3	Mappe di Karnaugh a 3 variabili	16
3.5.4	Esempi di mappe di Karnaugh a 4 variabili	17
3.6	Codifica binaria	18
3.7	Codice BCD	18
3.7.1	Esempio: display a 7 segmenti	18

3.8	Sommatori	19
3.8.1	Semi-sommatore (half adder)	19
3.8.2	Sommatore completo (full adder)	19
4	Sommatori, complemento a 2 e codici	21
4.1	Esempio: sommatore a 4 bit	21
4.2	Modularità e Design Gerarchico	21
4.3	Sottrattori e numeri negativi	22
4.3.1	Circuito sommatore/sottrattore a 4 bit	23
5	Codice Gray e ASCII	24
5.1	Codice Gray	24
5.1.1	Circuiti di conversione	24
5.1.2	Esercizi svolti: conversioni passo-passo	25
5.2	Codice ASCII	26
6	Logica sequenziale	28
6.1	Circuiti combinatori e circuiti sequenziali	28
6.2	Bistabilità e oscillazione	28
6.3	Latch	28
6.3.1	Latch RS con NOR: funzione caratteristica	29
6.3.2	Latch RS con abilitazione (Enable)	31
6.3.3	Latch di tipo D	31
6.3.4	Latch e Flip-Flop: differenza fondamentale	32
6.3.5	Tecnica master-slave	32
6.4	Tecnica trigger-edge	33
6.5	Ingressi sincroni e asincroni	34
6.6	JK Flip-Flop	34
6.6.1	Esempio pratico: il Registro come "filtro anti-glitch"	36
7	Registri a scorrimento e contatori	37
7.1	Registri a scorrimento	37
7.1.1	Modalità di trasmissione dati	37
7.1.2	Temporizzazione e vincoli	37
7.1.3	Schemi dei registri a scorrimento (SISO, SIPO, PISO)	38
7.2	Contatori	38
7.2.1	Contatore asincrono (ripple counter)	38
7.2.2	Contatore sincrono	39
7.2.3	Azzeramento dopo un certo conteggio (contatore modulo-M)	39
7.2.4	Ripple Carry Output (RCO)	39
8	Conteggio, divisori di frequenza e PRFG	41
8.1	Contatori a 8 bit: cascata di due SN74LS163	41
8.2	Divisori di frequenza per un numero n	41
8.2.1	Divisore con CLEAR sincrono	41
8.2.2	Divisore con SET sincrono (LOAD)	42
8.3	Contatore ad anello	42
8.4	Contatore ad anello <i>twisted</i> (Johnson counter)	43
8.5	Generatori di frequenza pseudo casuali (PRFG)	43

9	Macchine a stati finiti (FSM)	45
9.1	Tipi di architettura FSM: Moore e Mealy	45
9.1.1	Architettura di Moore	45
9.2	Dal diagramma di stato al circuito	45
9.2.1	Ricavare le funzioni dei segnali di uscita: $O_i = f(Q_i)$	46
9.2.2	Stato bloccante e segnale di Enable	46
9.2.3	Esempio: insegna luminosa (continuazione)	47
9.3	Architettura di Mealy	47
9.3.1	Tabella di verità dell'insegna luminosa (Mealy)	48
9.3.2	Tabella JK per l'implementazione	48
9.3.3	Mappe di Karnaugh per JK_0 e JK_1	48
9.4	Esempio: riconoscitore di sequenze	48
9.5	Esempio: macchina distributrice del caffè	49
9.5.1	Diagramma di stato della macchina del caffè (Moore)	49
10	Elettronica combinatoria complessa	50
10.1	ROM – Read Only Memory	50
10.1.1	Come è fatta?	50
10.1.2	Esempio: la tastiera	51
10.1.3	Esempio: la ROM del semaforo	51
10.1.4	Struttura della PLA: prima e dopo la programmazione	51
10.2	Multiplexer e demultiplexer	51
10.3	Universal Shift Register (USR)	52
10.3.1	Implementazione a 4 bit per trasmissione PISO	53
10.3.2	Applicazione: trasmissione seriale Sender–Receiver	53
10.4	Vincoli temporali dell'USR e Jitter	54
11	Conversione Digitale–Analogica e Analogica–Digitale	55
11.1	Conversione digitale $\leftrightarrow$ analogica	55
11.2	Caratteristiche del DAC	55
11.3	DAC a sommatore	56
11.4	DAC R-2R Ladder	56
11.5	Caratteristiche dell'ADC	57
12	Convertitori Analogico-Digitali (ADC)	58
12.1	ADC a gradinata	58
12.2	ADC ad approssimazioni successive (SAR)	58
12.3	Flash ADC	59
12.4	Osservazioni generali	59
12.5	ADC dell'AD2	59
12.5.1	ADC ad approssimazioni successive (SAR): analisi del circuito a 3 bit	59

Capitolo 1. Segnali, algebra di Boole e reti logiche

(27 febbraio – 2 marzo 2026)

1.1 Segnali analogici e digitali

Segnale analogico: assume valori continui (onde). È il tipo di segnale presente in natura.
Segnale digitale: assume valori discreti, tipicamente $\{0, 1\}$.

Il passaggio da un segnale analogico a uno digitale permette di **memorizzare meglio i dati**: un valore discreto è notevolmente più robusto al rumore, non subisce degrado nel tempo, ed è molto più semplice da elaborare e conservare in dispositivi di memoria rispetto a un segnale continuo. Inoltre, i sistemi digitali garantiscono una **riproducibilità esatta** dei risultati e godono di un'elevata flessibilità grazie alla possibilità di essere programmati via software.

1.2 Algebra di Boole

L'algebra di Boole fu pubblicata a metà dell'Ottocento (nel 1854, ad opera del matematico inglese George Boole, con il trattato *An Investigation of the Laws of Thought*) come base matematica per lo studio della logica. Per definirla formalmente servono i seguenti ingredienti:

- un insieme B che contiene **2 valori** (es. $\{0, 1\}$);
- su B agiscono **2 operazioni** (tipicamente $+$ e $\cdot$, cioè OR e AND);
- gli **elementi neutri** delle due operazioni sono gli stessi 2 elementi di B ;
- si definisce un'ulteriore operazione, il **NOT**, che manda i due valori l'uno nell'altro ($0 \rightarrow 1$, $1 \rightarrow 0$).

Notazione: la negazione di una variabile si indica con un trattino sopra la lettera, ad esempio $\overline{A}$.

1.2.1 Il concetto di duale

Duale di un'espressione booleana: si ottiene scambiando tra loro le due operazioni principali e gli elementi neutri:

- $+$ con $\cdot$;
- 0 con 1 ;
- il NOT resta invariato.

Quindi, se una proprietà è vera, anche la sua forma duale è vera.

Per esempio, il duale di

$$A + 0 = A$$

è

$$A \cdot 1 = A.$$

Allo stesso modo, il duale della proprietà di complementarietà

$$A + \overline{A} = 1$$

è

$$A \cdot \overline{A} = 0.$$

Il **principio di dualità** è molto utile perché, una volta nota una proprietà, si ottiene immediatamente la sua versione duale senza doverla dimostrare separatamente. Ad esempio:

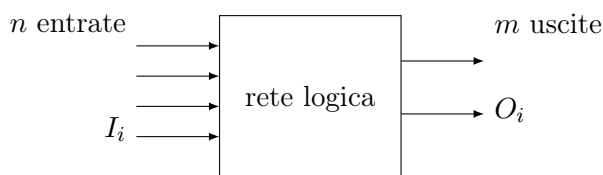
- commutativa: $A + B = B + A$ ha come duale $A \cdot B = B \cdot A$;
- associativa: $(A + B) + C = A + (B + C)$ ha come duale $(A \cdot B) \cdot C = A \cdot (B \cdot C)$;
- distributiva: $A + (B \cdot C) = (A + B) \cdot (A + C)$ ha come duale $A \cdot (B + C) = (A \cdot B) + (A \cdot C)$.

1.3 Cenni storici

- Nel 1938 **Claude Shannon** introdusse l'uso dell'algebra di Boole per l'analisi e la sintesi dei circuiti a relè, inventando di fatto l'applicazione alle reti logiche.
- **Huntington** (1904) formulò i famosi postulati che definiscono rigorosamente l'algebra booleana e dimostrò che ogni rete logica può essere implementata usando solo porte elementari.
- Il **transistor** (*transfer + resistor*) fu inventato nel 1947 ai Bell Labs. Da lì, nel giro di circa dieci anni, nacque l'elettronica a stato solido moderna e nel 1958 fu realizzato il primo circuito integrato.

1.4 Reti logiche

Una rete logica può essere vista come una scatola nera con n ingressi I_i e m uscite O_i :



$O_i = f(I_i)$: per ogni uscita esiste una funzione che dipende dagli ingressi I_i ; tale funzione è **suriettiva**. Sia gli O_i che gli I_i possono assumere solo i valori 0 oppure 1.

1.4.1 Funzioni a 1 ingresso

Con un solo ingresso I esistono $2^{2^1} = 4$ possibili funzioni di uscita:

I	O_1	O_2	O_3	O_4
0	0	1	0	1
1	0	0	1	1
	0	NOT	ID	1

Questa è la **tabella di verità** delle 4 funzioni a un ingresso: due sono costanti (0 e 1), una è l'**identità** (ID, o *buffer*) e una è la **negazione** (NOT).



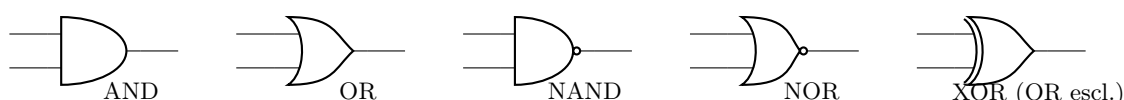
1.4.2 Funzioni a 2 ingressi

Con due ingressi I_1, I_0 ci sono $2^2 = 4$ combinazioni in ingresso e $2^{2^2} = 16$ possibili funzioni di uscita. La tabella completa è:

Funzione	00	01	10	11	Espressione Booleana	Nome comune / Significato
O_1	0	0	0	0	0	Costante 0 (Falsa)
O_2	1	0	0	0	$\overline{I_1} + \overline{I_0} = \overline{I_1} \cdot \overline{I_0}$	NOR
O_3	0	1	0	0	$\overline{I_1} \cdot I_0$	Inibizione di I_1 (da I_0)
O_4	0	0	1	0	$I_1 \cdot \overline{I_0}$	Inibizione di I_0 (da I_1)
O_5	0	0	0	1	$I_1 \cdot I_0$	AND
O_6	1	1	0	0	$\overline{I_1}$	NOT I_1
O_7	1	0	1	0	$\overline{I_0}$	NOT I_0
O_8	1	0	0	1	$\overline{I_1} \oplus I_0 = I_1 \odot I_0$	XNOR (Equivalenza)
O_9	0	1	1	0	$I_1 \oplus I_0$	XOR (OR esclusivo)
O_{10}	0	1	0	1	I_0	Identità I_0 (Buffer I_0)
O_{11}	0	0	1	1	I_1	Identità I_1 (Buffer I_1)
O_{12}	1	1	1	0	$\overline{I_1} \cdot I_0$	NAND
O_{13}	1	1	0	1	$I_1 + \overline{I_0} \text{ o } I_0 \rightarrow I_1$	Implicazione ($I_0 \implies I_1$)
O_{14}	0	1	1	1	$I_1 + I_0$	OR
O_{15}	1	0	1	1	$\overline{I_1} + I_0 \text{ o } I_1 \rightarrow I_0$	Implicazione ($I_1 \implies I_0$)
O_{16}	1	1	1	1	1	Costante 1 (Vera)

Nota sui simboli: L'operatore $\oplus$ (più cerchiato) rappresenta lo **XOR** (OR esclusivo, $I_1 \oplus I_0$), che vale 1 se e solo se i due ingressi sono diversi. L'operatore $\odot$ (punto cerchiato) rappresenta lo **XNOR** (equivalenza logica, $I_1 \odot I_0 = \overline{I_1} \oplus I_0$), che vale 1 se e solo se i due ingressi sono uguali.

Tra le 16 funzioni, le più importanti hanno un nome ed un simbolo proprio:



1.4.3 Funzioni a n ingressi

In generale, con n ingressi si hanno 2^n possibili combinazioni in ingresso e 2^{2^n} possibili funzioni di uscita. Le porte AND, NAND, OR, NOR, XOR si generalizzano naturalmente a n ingressi.

1.5 Bit di parità

Il **bit di parità** è un bit aggiuntivo utilizzato per rilevare errori singoli che possono verificarsi quando un segnale viene trasmesso in un canale rumoroso:

- Nella **parità pari** (*even parity*), il bit P si sceglie affinché il numero totale di “1” trasmessi sia pari:
 - se il numero di 1 nel dato è **pari** $\Rightarrow P = 0$;
 - se il numero di 1 nel dato è **dispari** $\Rightarrow P = 1$.

- Nella **parità dispari** (*odd parity*), la regola è invertita (il numero totale di “1” deve essere dispari).

Il controllo di parità è semplice ma limitato: può rilevare un numero dispari di errori (tipicamente un solo bit errato), ma fallisce se avvengono due errori simultanei, poiché la parità complessiva rimane inalterata.

Esempio numerico. Supponiamo di trasmettere il dato a 3 bit $D = 110$ usando **parità pari**:

- Il dato 110 contiene **due** 1 $\Rightarrow$ il numero di 1 è già pari $\Rightarrow P = 0$.
- Si trasmette la sequenza $\underbrace{1\ 1\ 0}_{\text{dato}}\ \underbrace{0}_P$ (totale: due 1, pari $\checkmark$).
- Se durante la trasmissione un bit si corrompe, ad es. arriva **1 0 0 0**: ora c'è solo un 1, il numero è **dispari** $\Rightarrow$ errore rilevato!
- Se invece si corrompono **due** bit contemporaneamente, la parità resta pari e l'errore **non** viene rilevato (limite del metodo).

1.6 Esercizio: semplice circuito per abilitare la presa dati

Disegnare un circuito che generi un segnale per abilitare la presa dati di un esperimento se ha il segnale di **sicurezza** S attivo, se c'è la richiesta di una presa dati che può essere di **calibrazione** o **standard** (non entrambe), e se l'operatore chiede di abilitare l'**acquisizione**. Inoltre la presa dati si deve arrestare se S non è più attivo.

Come si risolve (ragionamento passo per passo):

1. **Identificare le condizioni.** L'uscita deve essere 1 (abilitata) *solo* quando **tutte** le seguenti condizioni sono soddisfatte:

- il segnale di sicurezza $S = 1$;
- il segnale di acquisizione $A = 1$;
- esattamente **uno** tra calibrazione C e standard T è attivo (non né entrambi, né nessuno).

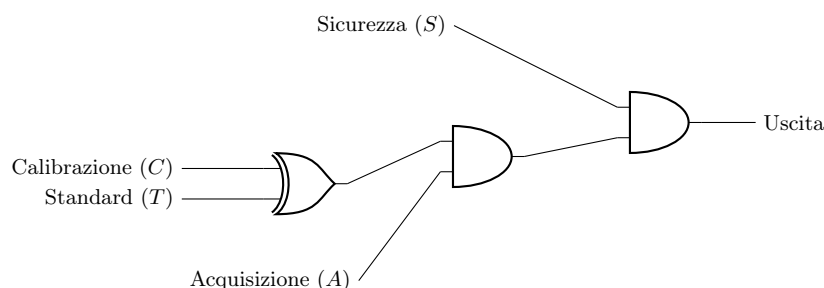
2. **Tradurre ogni condizione in logica:**

- “Sicurezza attiva” $\rightarrow$ basta S .
- “Acquisizione attiva” $\rightarrow$ basta A .
- “Calibrazione o Standard, ma **non entrambe**” $\rightarrow$ questa è esattamente la definizione dello **XOR**: $C \oplus T$ vale 1 quando i due ingressi sono *diversi* (uno 0 e uno 1), e 0 quando sono uguali (entrambi 0 oppure entrambi 1).

3. **Combinare le condizioni con AND:** poiché tutte e tre le condizioni devono essere vere *contemporaneamente*, si mettono in AND:

$$\text{Uscita} = S \cdot A \cdot (C \oplus T)$$

4. **Ordine delle porte:** si mette S nell'ultima porta AND così, quando il segnale di sicurezza si abbassa, l'uscita si spegne immediatamente (minimo ritardo di propagazione).



Verifica con un esempio: supponiamo $S = 1$, $A = 1$, $C = 1$, $T = 0$.

$$C \oplus T = 1 \oplus 0 = 1, \quad \text{Uscita} = 1 \cdot 1 \cdot 1 = 1 \quad \checkmark$$

Se invece sia calibrazione che standard sono attivi ($C = 1$, $T = 1$): $C \oplus T = 0 \Rightarrow \text{Uscita} = 0$ (corretto: le due richieste si escludono a vicenda).

Capitolo 2. Realizzazione fisica delle porte logiche

(6 marzo 2026)

2.1 Esercizio: bit di parità a 4 bit

Progettare un circuito che calcoli il bit di parità di un numero a 4 bit. Il bit di parità è 0 se il numero di 1 nel numero è pari, 1 se è dispari.

Come si risolve (ragionamento passo per passo):

1. **Capire cosa serve (e perché può sembrare invertita).** L'obiettivo della **parità pari** è che il numero totale di 1 trasmessi (dato + bit P) sia sempre **pari**. Di conseguenza:

- se nel dato ci sono già un numero **pari** di 1 $\Rightarrow$ non serve aggiungere nulla $\Rightarrow P = 0$;
- se nel dato ci sono un numero **dispari** di 1 $\Rightarrow$ bisogna aggiungere un 1 per bilanciare $\Rightarrow P = 1$.

Attenzione: $P = 1$ **non** significa “va tutto bene”, significa “ho dovuto correggere la parità”. Detto in modo diretto: P vale 1 quando il conteggio degli 1 nel dato è **dispari** (e dunque serviva un 1 extra per renderlo pari).

2. **Riconoscere la porta giusta.** Ricordiamo la tabella dello **XOR** a 2 ingressi:

A	B	$A \oplus B$	Significato
0	0	0	zero uni $\Rightarrow$ pari
0	1	1	un uno $\Rightarrow$ dispari
1	0	1	un uno $\Rightarrow$ dispari
1	1	0	due uni $\Rightarrow$ pari

Lo XOR vale 1 esattamente quando il numero di 1 in ingresso è **dispari**: è dunque il componente perfetto per calcolare la parità!

3. **Estendere a 4 bit con la proprietà associativa.** Lo XOR è **associativo**, quindi possiamo calcolare la parità su più bit applicandolo in cascata, come somma modulo 2:

$$P = I_3 \oplus I_2 \oplus I_1 \oplus I_0$$

Il risultato vale 1 se e solo se il numero totale di 1 tra i 4 bit è dispari (esattamente ciò che vogliamo).

4. **Costruire il circuito.** Uno XOR accetta solo 2 ingressi, quindi si procede **a coppie**:

- Primo livello: $X_A = I_1 \oplus I_0$ e $X_B = I_3 \oplus I_2$ (due porte XOR in parallelo).
- Secondo livello: $P = X_A \oplus X_B$ (una porta XOR che combina i risultati parziali).

Servono quindi **3 porte XOR** in totale.

5. **Verifica numerica.** Prendiamo $N = 1011$:

- Conta degli uni: tre 1 $\Rightarrow$ dispari $\Rightarrow$ ci aspettiamo $P = 1$.
- Calcolo: $X_A = 1 \oplus 1 = 0$, $X_B = 1 \oplus 0 = 1$, $P = 0 \oplus 1 = 1 \checkmark$

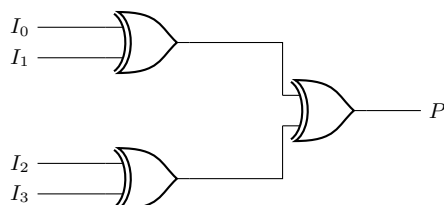
Prendiamo $N = 1100$:

- Conta degli uni: due 1 $\Rightarrow$ pari $\Rightarrow$ ci aspettiamo $P = 0$.
- Calcolo: $X_A = 0 \oplus 0 = 0$, $X_B = 1 \oplus 1 = 0$, $P = 0 \oplus 0 = 0 \checkmark$

Sia $N = I_3 I_2 I_1 I_0$. Si lavora a coppie di bit, applicando lo XOR a multipli di 2:

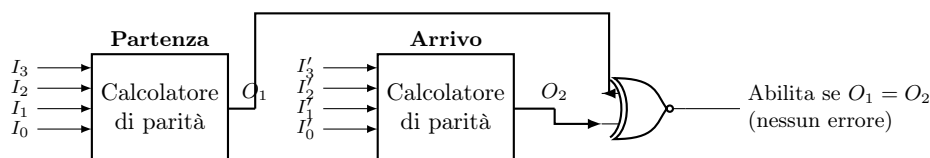
I_3	I_2	XOR	I_1	I_0	XOR
0	0	0	0	0	0
0	1	1	0	1	1
1	0	1	1	0	1
1	1	0	1	1	0

Il **calcolatore di parità** si ottiene combinando in cascata 3 porte XOR:



2.2 Verifica di una trasmissione tramite parità

Calcolando il bit di parità sia alla **partenza** (O_1) che all'**arrivo** (O_2) e confrontandoli con una porta **XNOR**, si fa passare il segnale solo se $O_1 = O_2$ (cioè 00 oppure 11): questo permette di controllare che il segnale non sia stato alterato durante la trasmissione.



2.3 Caratteristiche fisiche delle porte logiche

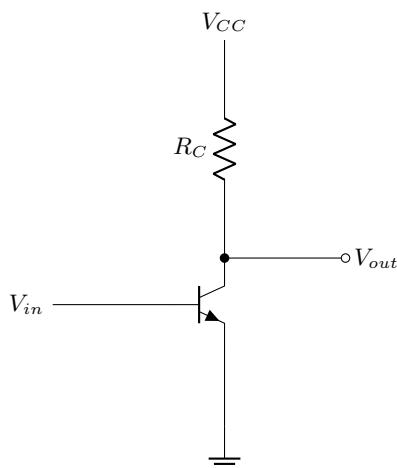
Le diverse **famiglie logiche** (RTL, DTL, HTL, TTL, ECL, MOS, CMOS) si confrontano in base a parametri come la porta di base, il fan-out, la potenza dissipata, l'immunità al rumore, il ritardo di propagazione e la frequenza massima di funzionamento. In generale:

- **TTL** (Transistor-Transistor Logic): ottimo compromesso velocità/potenza, molto diffusa;
- **ECL** (Emitter-Coupled Logic): velocissima ma con elevato consumo;
- **CMOS** (Complementary Metal-Oxide-Semiconductor): bassissima potenza dissipata, oggi la più usata nei circuiti integrati.

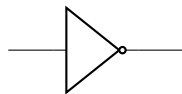
2.4 Implementazione delle porte logiche

2.4.1 NOT con un singolo transistor

La porta NOT più semplice si realizza con un singolo transistor NPN in configurazione a emettitore comune, con una resistenza di pull-up R_C verso V_{CC} :



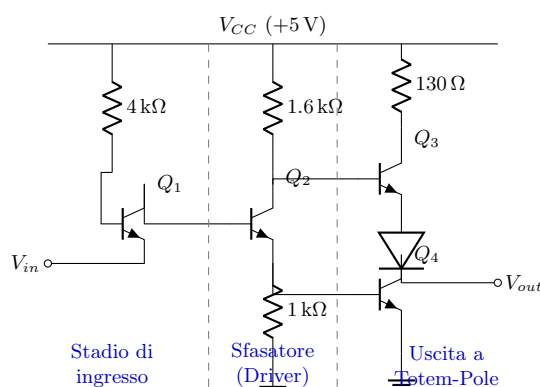
- $V_{in} = 5\text{ V} (= 1) \Rightarrow$ il transistor satura $\Rightarrow V_{out} = 0,2\text{ V} (= 0)$;
- $V_{in} = 0\text{ V} (= 0) \Rightarrow$ il transistor è spento $\Rightarrow V_{out} = 5\text{ V} (= 1)$.



2.4.2 NOT in tecnologia TTL

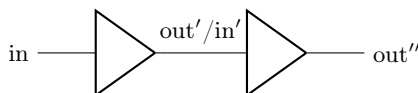
Nella tecnologia TTL (*Transistor-Transistor Logic*), la porta logica base non utilizza singoli diodi o resistenze in ingresso, ma sfrutta i transistor bipolari (BJT) per migliorare velocità e prestazioni. La porta NOT TTL standard si divide in tre stadi principali:

1. **Stadio di ingresso:** utilizza un transistor NPN (Q_1) in cui l'emettitore funge da ingresso. Se l'ingresso è basso (0 V), Q_1 va in conduzione portando a massa la base dello stadio successivo. Se l'ingresso è alto (5 V), la giunzione base-emettitore è polarizzata inversamente e la corrente fluisce verso la base del driver.
2. **Sfasatore (driver):** il transistor centrale (Q_2) riceve il segnale da Q_1 e pilota in controfase i due transistor finali. Produce due segnali logici opposti: uno al collettore e uno all'emettitore.
3. **Stadio di uscita a totem-pole:** costituito da due transistor (Q_3 e Q_4) impilati uno sull'altro e un diodo. Garantisce un'uscita "forte" a bassa impedenza in entrambi gli stati logici: Q_3 spinge attivamente l'uscita a livello alto (pull-up attivo), mentre Q_4 la tira a massa (pull-down). Il diodo serve a garantire che Q_3 e Q_4 non conducano mai contemporaneamente in fase di transizione.



2.5 Immunità al rumore

L'immunità al rumore di un sistema digitale non è totale, ma è comunque molto più elevata di quella dei sistemi analogici, perché due porte in cascata "ripuliscono" il segnale ad ogni passaggio rigenerando i livelli di tensione:



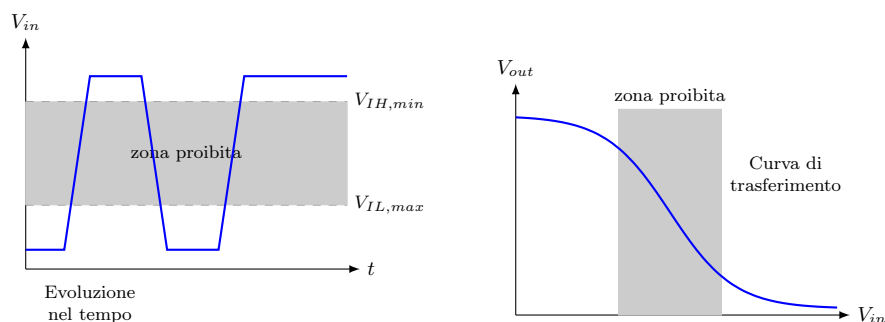
Per garantire il corretto riconoscimento dei livelli logici, i costruttori definiscono le soglie di tensione per l'ingresso e l'uscita:

Livello alto (High)	Livello basso (Low)
• $V_{OH,min}$: V_{out} minima garantita per l'uscita alta; • $V_{OH,max}$: V_{out} massima possibile per l'uscita alta; • $V_{IH,min}$: V_{in} minima richiesta in ingresso per essere letta come alta; • $V_{IH,max}$: V_{in} massima ammessa per un ingresso alto.	• $V_{OL,min}$: V_{out} minima possibile per l'uscita bassa; • $V_{OL,max}$: V_{out} massima garantita per l'uscita bassa; • $V_{IL,min}$: V_{in} minima ammessa per un ingresso basso; • $V_{IL,max}$: V_{in} massima tollerata in ingresso per essere letta come bassa.

La differenza tra le tensioni garantite in uscita e quelle minime/massime tollerate in ingresso definisce i **margini di rumore** (*noise margin*):

$$NM_H = V_{OH,min} - V_{IH,min} \quad NM_L = V_{IL,max} - V_{OL,max}$$

I margini di rumore (NM_H e NM_L) rappresentano l'ampiezza massima di un disturbo (rumore elettrico, fluttuazioni) che può sommarsi al segnale durante il tragitto lungo il filo senza far sì che la porta successiva lo interpreti in modo errato o lo collochi nella *zona proibita*. In altre parole, è il “cuscinetto di sicurezza” contro il rumore.



Cosa rappresentano i due grafici:

- **Grafico di sinistra (Evoluzione nel tempo):** Mostra come varia nel tempo la tensione in ingresso (V_{in}) ad una porta. La fascia grigia è la **zona proibita**: se la tensione cade in quest'area, il componente *non può* stabilire se il livello logico sia uno 0 o un 1. I fronti di salita e discesa del segnale devono attraversare questa zona il più rapidamente possibile.
- **Grafico di destra (Curva di trasferimento):** È il grafico V_{out} in funzione di V_{in} (es. per un inverter NOT). Un inverter ideale commuterebbe a gradino esatto, ma nella realtà c'è una pendenza. Mostra chiaramente che finché V_{in} resta *fuori* dalla zona proibita grigia (a sinistra o a destra), V_{out} assume un valore netto e stabile ai margini del grafico logico.

2.6 Nomenclatura delle famiglie logiche e fan-out

2.6.1 Nomenclatura dei componenti

Le sigle dei circuiti integrati logici (IC) commerciali seguono uno schema di identificazione ben preciso. Ad esempio, una porta **SN74LS00N** viene così codificata:

$$\underbrace{SN}_{\text{ditta produttrice}} \quad \underbrace{74}_{\text{famiglia logica}} \quad \underbrace{LS}_{\text{sotto-famiglia}} \quad \underbrace{00}_{\text{funzione}} \quad \underbrace{N}_{\text{package}}$$

In dettaglio:

- **Ditta produttrice:** ad esempio *SN* per Texas Instruments, *MC* per Motorola.
- **Famiglia logica / range temperature:** la famiglia **74** è per uso commerciale ($0^\circ\text{C} \div 70^\circ\text{C}$), mentre la **54** è per uso militare/aerospaziale con tolleranze molto più ampie ($-55^\circ\text{C} \div 125^\circ\text{C}$).
- **Sotto-famiglia (tecnologia):** specifica le caratteristiche di velocità e consumi. Ad esempio *LS* (Low-power Schottky), *HC* (High-speed CMOS), *ALS* (Advanced Low-power Schottky).
- **Funzione:** il codice *00* indica ad esempio “Quad 2-input NAND gate” (4 porte NAND a due ingressi), *04* indica un inverter NOT (esadimensionale), ecc.
- **Package:** indica il tipo di contenitore fisico del chip. Ad esempio la lettera *N* indica il classico package plastico DIP (Dual In-line Package) per fori passanti, *D* indica un package SOIC per montaggio superficiale.

2.6.2 Fan-Out (capacità di pilotaggio)

Fan-Out = numero massimo di ingressi di porte logiche uguali che l'uscita di una determinata porta è in grado di pilotare, garantendo i corretti livelli di tensione.

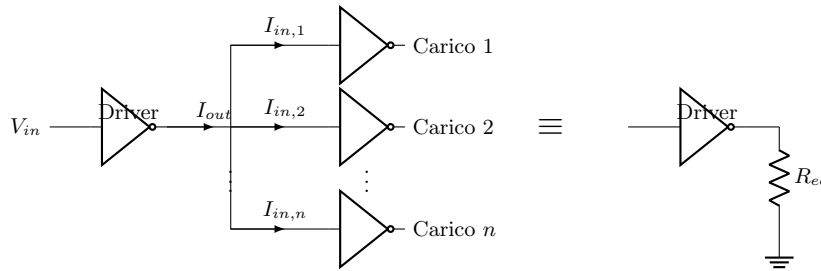
Perché c'è un limite? Ogni porta in ingresso assorbe (o eroga) una piccolissima quantità di corrente. Se colleghiamo troppe porte all'uscita di un'unica porta “driver”, la somma di queste piccole correnti diventa elevata.

Se il driver non è progettato per erogare tutta questa corrente, la sua tensione di uscita inizia ad abbassarsi (o alzarsi) finendo nella zona proibita e causando errori nel circuito.

Il calcolo del Fan-Out si effettua separatamente per i due stati logici Alto e Basso, prendendo poi il caso peggiore (il minimo tra i due calcoli):

$$FO_H = \frac{I_{out,H}}{I_{in,H}} \qquad FO_L = \frac{I_{out,L}}{I_{in,L}}$$

Nello schema seguente è illustrato il concetto: la corrente totale I_{out} che fuoriesce dalla porta “Driver” si divide in tante piccole I_{in} quanti sono i carichi. Tutta la rete di carichi può essere modellata teoricamente come un’unica resistenza equivalente verso massa o verso l’alimentazione:



Capitolo 3. Algebra di Boole: assiomi, proprietà e sintesi

(13 marzo – 16 marzo 2026)

3.1 Tempi caratteristici delle porte logiche

Tempo di propagazione t_P : calcolato fra V_{in} e V_{out} ; è il tempo che intercorre fra i momenti in cui i segnali raggiungono il 50% della loro intensità fra V_{OL} e V_{OH} . È dovuto alle risposte dei transistor (inter-sat).

Tempo di transizione t_r : calcolato solo fra V_{in} e V_{out} ; è il tempo fra i momenti in cui il segnale è al 10% e al 90% (salita) o al 90% e al 10% (discesa). È dovuto ad effetti capacitivi.

3.2 Assiomi dell'algebra di Boole

L'algebra di Boole è definita dai seguenti assiomi su un insieme B con $|B| = 2$:

- **Chiusura:** $\forall a, b \in B \Rightarrow a + b \in B$ (OR) e $a \cdot b \in B$ (AND).
- **Elementi neutri:** $a + 0 = a$ (0 è neutro per OR); $a \cdot 1 = a$ (1 è neutro per AND).
- **Operazioni complemento:** $\bar{a} + a = 1$ e $\bar{a} \cdot a = 0$ (elemento opposto).
- **Involuzione:** $\bar{\bar{a}} = a$.

3.3 Proprietà dell'algebra di Boole

- **Commutativa:** $a + b = b + a$; $a \cdot b = b \cdot a$.
- **Associativa:** $(a + b) + c = a + b + c$; $(a \cdot b) \cdot c = a \cdot b \cdot c$.
- **Distributiva:** $(a + b) \cdot c = a \cdot c + b \cdot c$; $(a \cdot b) + c = (a + c) \cdot (b + c)$.
Si verifica con le tabelle di verità.
- **Dualità:** posso sempre scambiare un'operazione con l'altra insieme a tutti gli elementi (operazione, neutro, opposto). Funziona perché è un'algebra a 2 elementi, quindi 0 è uno e 1 è l'altro: $a + 0 = a \Rightarrow a \cdot 1 = a$.
- **Assorbimento:** $a + a \cdot b = a$.
- **Idempotenza:** $a + a = a$; $a \cdot a = a$.

3.3.1 Teorema della semplificazione

$$x \cdot y + x \cdot \bar{y} = x$$

$$x \cdot y + x \cdot \bar{y} = x \cdot (y + \bar{y}) = x \cdot 1 = x$$

Duale: $(x + y) \cdot (\bar{x} + y) = x \cdot \bar{x} + xy + \bar{x}y + y = 0 + y + y = y$.

3.3.2 Teorema della moltiplicazione e fattorizzazione

$$(x + y) \cdot (\bar{x} + z) = x \cdot z + \bar{x} \cdot y$$

Dimostrazione:

$$(x + y) \cdot (\bar{x} + z) = \bar{x} \cdot x + x \cdot z + \bar{x} \cdot y + y \cdot z = x \cdot z + \bar{x} \cdot y + yz(x + \bar{x}) = x \cdot z(1 + y) + \bar{x} \cdot y(1 + z) = x \cdot z + \bar{x} \cdot y$$

(usando $OR Y = 1 \forall y = 0, 1$)

Duale: $(\bar{x} \cdot z) + (x \cdot y) = (\bar{x} + y) \cdot (x + z)$.

3.3.3 Leggi di De Morgan

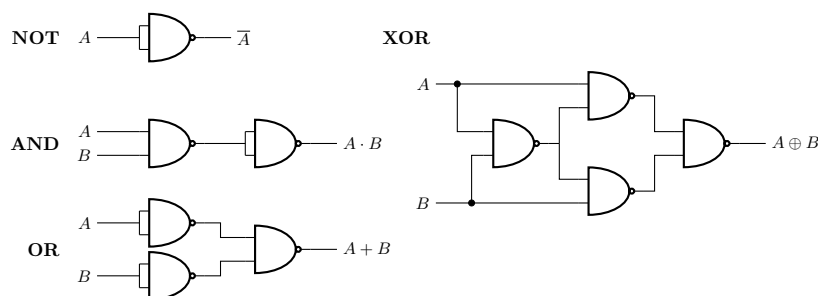
$$x + y + z + \dots = \overline{\overline{x} \cdot \overline{y} \cdot \overline{z} \cdot \dots}$$

$$x \cdot y \cdot z \cdot \dots = \overline{\overline{x} + \overline{y} + \overline{z} + \dots}$$

3.4 Porte universali: NAND e NOR

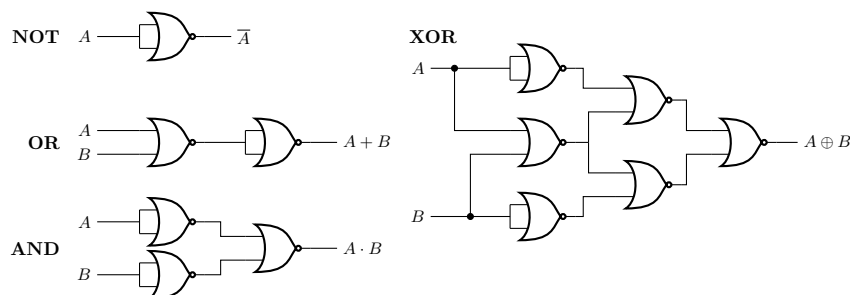
Le porte NAND e NOR sono dette **porte universali** (o un set funzionalmente completo) perché con ciascuna di esse è possibile realizzare *qualsiasi* funzione booleana, replicando il comportamento di NOT, AND e OR. Al contrario, AND e OR *non* formano insieme universali da soli, poiché, non potendo invertire un segnale, non permettono di realizzare la funzione NOT.

3.4.1 NAND come porta universale



3.4.2 NOR come porta universale

In modo duale alla NAND, anche la NOR può realizzare NOT, AND, OR e XOR:



3.5 Dalla tabella di verità al circuito

Data una tabella di verità, possiamo ricavare le espressioni logiche della funzione e i relativi circuiti. Consideriamo la seguente tabella per una funzione $F(a, b, c)$:

a	b	c	$F(a, b, c)$
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1

Dalle **possibili uscite di F** possiamo ricavare le due forme canoniche attraverso due procedimenti duali. Vediamoli passo passo in modo semplice:

1. Mintermini e Prima Forma Canonica (SOP)

La logica è “costruire le combinazioni che ci danno 1”:

- **Quali righe guardare:** Si prendono in considerazione solo le righe in cui $F = 1$.
- **Come scrivere il termine:** Per ogni riga, scriviamo il **prodotto (AND)** delle variabili.
- **Regola della negazione:** Se nella tabella la variabile vale **0** la scriviamo **negata** (con la barretta), se vale **1** la scriviamo **normale**.
- **Risultato finale:** Si sommano (OR) tra loro tutti i prodotti ottenuti. Si crea così la **SOP** (*Sum Of Products* - Somma di Prodotti).

Guardando la tabella precedente:

$$F(a, b, c) = \underbrace{\bar{a}bc}_{\text{riga } 0,0,1} + \underbrace{\bar{a}b\bar{c}}_{\text{riga } 0,1,0} + \underbrace{a\bar{b}\bar{c}}_{\text{riga } 1,0,0} + \underbrace{ab\bar{c}}_{\text{riga } 1,1,0} + \underbrace{abc}_{\text{riga } 1,1,1}$$

2. Maxtermini e Seconda Forma Canonica (POS)

La logica qui è “escludere le combinazioni che ci danno 0”:

- **Quali righe guardare:** Si prendono in considerazione solo le righe in cui $F = 0$.
- **Come scrivere il termine:** Per ogni riga, scriviamo la **somma (OR)** delle variabili.
- **Regola della negazione (qui si inverte!):** Se nella tabella la variabile vale **1** la scriviamo **negata**, se vale **0** la scriviamo **normale**.
- **Risultato finale:** Si moltiplicano (AND) tra loro tutte le somme ottenute. Si crea così la **POS** (*Product Of Sums* - Prodotto di Somme).

Guardando la tabella precedente:

$$F(a, b, c) = \underbrace{(a + b + c)}_{\text{riga } 0,0,0} \cdot \underbrace{(a + \bar{b} + \bar{c})}_{\text{riga } 0,1,1} \cdot \underbrace{(\bar{a} + b + \bar{c})}_{\text{riga } 1,0,1}$$

Nota: Di solito si semplificano queste espressioni (es. con le mappe di Karnaugh) prima di costruire i circuiti per risparmiare componenti.

3.5.1 Equivalenza delle due forme canoniche

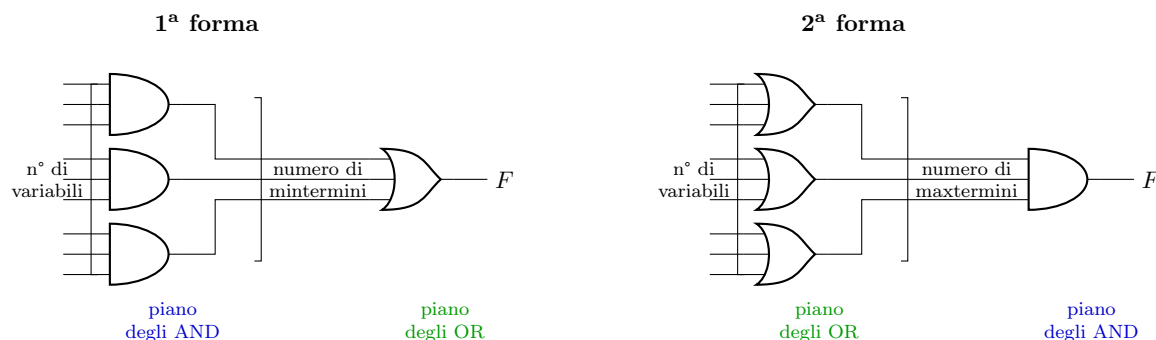
Dimostriamo che le due forme canoniche descrivono la stessa funzione e sono equivalenti. Prendiamo i mintermini per cui $F = 0$ (cioè m_0, m_3, m_5) e neghiamo la loro somma, ovvero calcoliamo $\overline{F}$. Per le leggi di De Morgan otteniamo esattamente la seconda forma canonica (POS):

$$\begin{aligned} \overline{\bar{a}\bar{b}\bar{c} + \bar{a}bc + abc} &= \text{(per le leggi di De Morgan)} \\ &= \overline{(\bar{a}\bar{b}\bar{c}) \cdot (\bar{a}bc) \cdot (abc)} \\ &= (a + b + c) \cdot (a + \bar{b} + \bar{c}) \cdot (\bar{a} + b + \bar{c}) \end{aligned}$$

Questo dimostra che le due forme canoniche portano allo stesso identico risultato logico.

3.5.2 I circuiti che corrispondono alle due forme

A queste due forme canoniche corrispondono due tipi di circuiti, entrambi strutturati su due "piani" di porte logiche. Il numero di ingressi e uscite dei piani dipende dal numero di variabili e dal numero di mintermini (o maxtermini):



3.5.3 Mappe di Karnaugh a 3 variabili

Data la tabella di verità vista in precedenza, possiamo costruire la mappa di Karnaugh a 3 variabili $c \backslash ab$. Le colonne sono ordinate secondo il **codice Gray** (00, 01, 11, 10). Questo ordine è fondamentale perché garantisce che le celle fisicamente adiacenti differiscano per **un solo bit**. In questo modo, raggruppando gli uni (o gli zeri) vicini, possiamo eliminare le variabili che cambiano di stato e ottenere un'espressione semplificata!

Semplificazione con Mintermini (uni)

$c \backslash ab$	00	01	11	10
0	0	1	1	1
1	1	0	1	0

Analisi dettagliata: Mintermini (Forma SOP)

L'obiettivo della mappa è **semplificare una funzione logica** raggruppando gli **1**. La regola d'oro per trovare la formula di ogni blocco è: se una variabile **cambia valore** all'interno del gruppo **si elimina**. Se **rimane costante**, **si tiene** (normale se 1, negata se 0).

- **Il blocco ROSSO (verticale al centro):** Le colonne (ab) sono sempre **11**, quindi $a = 1$ e $b = 1$ non cambiano e **le teniamo entrambe** (ab). La riga (c) passa da 0 a 1, quindi cambia e **la eliminiamo**. Risultato: ab .
- **Il blocco BLU (orizzontale in alto a sinistra):** La riga (c) è sempre **0**, quindi **la teniamo negata** ($\bar{c}$). Le colonne (ab) passano da 01 a 11: la a cambia (da 0 a 1) e si elimina, la b resta 1 e si tiene. Risultato: $b\bar{c}$.
- **Il blocco VERDE (orizzontale in alto a destra):** La riga (c) è sempre **0**, quindi **la teniamo negata** ($\bar{c}$). Le colonne (ab) passano da 11 a 10: la a resta 1 e si tiene, la b cambia (da 1 a 0) e si elimina. Risultato: $a\bar{c}$.
- **Il blocco ARANCIO (cella isolata in basso):** Non ha vicini, nessuna variabile cambia. Le teniamo tutte e tre per il valore che hanno: riga $c = 1$, colonna $ab = 00$. Risultato: $\bar{a}\bar{b}c$.

Unendo tutti i risultati parziali con un segno + (OR), otteniamo:

$$F = ab + b\bar{c} + a\bar{c} + \bar{a}\bar{b}c$$

Semplificazione con Maxtermini (zeri)

$c \backslash ab$	00	01	11	10
0	0	1	1	1
1	1	0	1	0

Analisi dettagliata: Maxtermini (Forma POS)

Se con i mintermini cercavamo gli 1, qui ci concentriamo **solo sugli 0**. La logica si **ribalta completamente**: i valori si invertono (se vale 0 si scrive normale a , se vale 1 si scrive negata $\bar{a}$) e le operazioni si invertono (somme dentro le parentesi, moltiplicazioni fuori).

In questa mappa specifica, gli zeri si trovano in diagonale. Poiché non sono vicini in orizzontale o verticale, **sono isolati** e non possiamo raggrupparli. Li prendiamo uno per uno:

- **Lo zero MARRONE (in alto a sinistra):** Riga $c = 0$, colonna $ab = 00$ (quindi $a = 0, b = 0$). Applicando il “mondo al contrario”, diventano tutti normali e li uniamo con il +. Risultato: $(a + b + c)$.
- **Lo zero VIOLA (in basso al centro):** Riga $c = 1$, colonna $ab = 01$ (quindi $a = 0, b = 1$). I valori a 1 si negano. Risultato: $(a + \bar{b} + \bar{c})$.
- **Lo zero AZZURRO (in basso a destra):** Riga $c = 1$, colonna $ab = 10$ (quindi $a = 1, b = 0$). I valori a 1 si negano. Risultato: $(\bar{a} + b + \bar{c})$.

Moltiplichiamo (AND) tra loro i tre blocchi ottenuti per la soluzione finale:

$$F = (a + b + c) \cdot (a + \bar{b} + \bar{c}) \cdot (\bar{a} + b + \bar{c})$$

3.5.4 Esempi di mappe di Karnaugh a 4 variabili

Anche per 4 variabili si usa la codifica di Gray per disporre le combinazioni di due variabili (ab e cd) sui lati della tabella. I raggruppamenti (implicanti) devono essere composti da un numero di “1” (o “0”) pari a una potenza di due (1, 2, 4, 8...) e formare rettangoli. Essi possono anche “sconfinare” da un bordo all’altro della mappa, poiché i bordi opposti sono adiacenti.

L’obiettivo è coprire tutti gli 1 (o gli 0) col minor numero di raggruppamenti, il più larghi possibile. Analizziamo due funzioni diverse usando le mappe dei tuoi appunti originali.

Mappa 1: Quando usare i Maxtermini NON conviene

$ab \backslash cd$	00	01	11	10
00	0	0	0	0
01	0	1	0	0
11	0	0	1	0
10	0	0	0	0

Analisi della Mappa 1: Perché i Maxtermini falliscono

In questa prima funzione, abbiamo ben quattordici zeri e solo due uni. Se proviamo a usare i **Maxtermini** (raggruppando gli 0, logica inversa), l’alto numero di zeri ci costringe a fare ben **4 raggruppamenti** che si accavallano caoticamente sfruttando anche i bordi!

- **Bordi orizzontali VIOLA (8 celle):** Comprende le righe $cd = 00, 10$. Resta solo $d = 0 \rightarrow d$. Risultato: (d) .
- **Bordi verticali MARRONI (8 celle):** Comprende le colonne $ab = 00, 10$. Resta solo $b = 0 \rightarrow b$. Risultato: (b) .
- **Quadrato AZZURRO (4 celle, alto a dx):** Colonne 11, 10 ($a = 1 \rightarrow \bar{a}$) e righe 00, 01 ($c = 0 \rightarrow c$). Risultato: $(\bar{a} + c)$.
- **Quadrato MAGENTA (4 celle, basso a sx):** Colonne 00, 01 ($a = 0 \rightarrow a$) e righe 11, 10 ($c = 1 \rightarrow \bar{c}$). Risultato: $(a + \bar{c})$.

Espressione trovata: $F = (d) \cdot (b) \cdot (\bar{a} + c) \cdot (a + \bar{c})$

Se invece usassimo i **Mintermini** per la stessa mappa, ci basterebbe raggruppare i due singoli **1** isolati (2 gruppi da una cella). Si otterrebbe $F = \bar{a}b\bar{c}d + abcd$ (solo **due** termini). **Conclusione:** quando gli zeri sono troppi, usare i Maxtermini porta a un’espressione enorme.

Mappa 2: Un buon uso dei Mintermini

$ab \backslash cd$	00	01	11	10
00	0	0	0	0
01	1	1	1	1
11	0	1	1	0
10	0	0	0	0

Analisi della Mappa 2 con i Mintermini

In questa seconda funzione abbiamo meno uni (sei) che zeri (dieci). Ci conviene usare i **Mintermini** e raggruppare gli 1:

- **Rettangolo BLU (4 celle, riga intera):** Copre tutta la riga $cd = 01$. Quindi $c = 0 \rightarrow \bar{c}$ e $d = 1 \rightarrow d$. Le colonne variano tutte, quindi si eliminano a e b . Risultato: $\bar{c}d$.

- **Quadrato ROSSO (4 celle, centrale):** Copre le colonne $ab = 01, 11$ (a cambia, resta $b = 1 \rightarrow b$) e le righe $cd = 01, 11$ (c cambia, resta $d = 1 \rightarrow d$). Risultato: bd .

L'espressione ottimizzata risultante è semplicemente la somma dei due gruppi:

$$F = \overline{c}d + bd = (\overline{c} + b)d$$

3.6 Codifica binaria

Con n bit posso descrivere 2^n valori in decimale.

Esempio: $N = 13 \leftrightarrow 1101$.

Conversione binario $\rightarrow$ decimale: si moltiplica ogni cifra per la potenza di 2 corrispondente alla sua posizione e si somma:

$$1101 \rightarrow 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 8 + 4 + 0 + 1 = 13$$

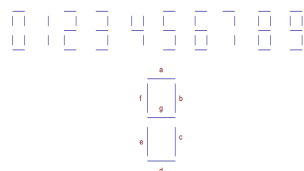
Conversione decimale $\rightarrow$ binario: si divide ripetutamente per 2, prendendo via via i resti dal basso verso l'alto:

$$\begin{aligned} 13 \rightarrow \frac{13}{2} = 6 \text{ resto } 1 \rightarrow \frac{6}{2} = 3 \text{ resto } 0 \rightarrow \frac{3}{2} = 1 \text{ resto } 1 \rightarrow \frac{1}{2} = 0 \text{ resto } 1 \\ \Rightarrow 13 = 1101 \text{ (leggendo i resti dal basso verso l'alto)} \end{aligned}$$

3.7 Codice BCD

BCD (Binary Coded Decimal): per scrivere le cifre decimali da 0 a 9 si usano 4 bit (3 bit sarebbero troppo pochi, dato che $2^3 = 8 < 10$).

3.7.1 Esempio: display a 7 segmenti



Si vuole pilotare un singolo segmento di un display a 7 segmenti a partire dalla cifra BCD in ingresso a, b, c, d .

Si costruisce la tabella di verità del segmento (con $a \leftrightarrow$ bit più significativo, $d \leftrightarrow$ bit meno significativo) e si segnano come **don't care** ($\times$) le combinazioni che non corrispondono a nessuna cifra BCD (cioè da 10 a 15), in modo da poter scegliere il loro valore così da semplificare il circuito:

a	b	c	d	uscita
0	0	0	0	0
0	0	0	1	1
0	0	1	0	0
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1010 – 1111				$\times \times \times \times \times \times$

La 2^a forma canonica (POS) del segmento, prima della semplificazione, è:

$$F(a, b, c, d) = (a + b + c + d) \cdot (a + b + c + \overline{d}) \cdot (a + \overline{b} + \overline{c} + \overline{d})$$

Riportando la funzione (con i don't care) sulla mappa di Karnaugh a 4 variabili $cd \backslash ab$ e scegliendo opportunamente il valore dei don't care, si semplifica notevolmente il circuito:

$cd \backslash ab$	00	01	11	10
00	0	1	×	0
01	1	0	×	×
11	0	1	×	×
10	0	0	×	×

Scegliendo i don't care in modo da formare i raggruppamenti più grandi possibile, l'espressione semplificata diventa:

$$a + c\bar{d} + \bar{b}c + b\bar{c} = a + c\bar{d} + (b \oplus c)$$

molto più corta e immediata da realizzare rispetto alla forma canonica di partenza.

I **don't care** si scelgono in modo da semplificare il più possibile il circuito risultante.

3.8 Sommatore

3.8.1 Semi-sommatore (half adder)

Un **semi-sommatore** (half adder) è un circuito combinatorio in grado di eseguire la somma di due singoli bit (a_i e b_i). Esso produce due uscite: il bit di somma Σ_i e il bit di riporto generato C_{i+1} (carry out). L'equazione logica che descrive la somma è un'operazione XOR ($\Sigma_i = a_i \oplus b_i$), mentre il riporto è dato da un'operazione AND ($C_{i+1} = a_i \cdot b_i$). Il limite del semi-sommatore è che non prevede un ingresso per il riporto proveniente da uno stadio precedente, limitandone l'utilizzo al solo bit meno significativo di un'addizione su più bit.

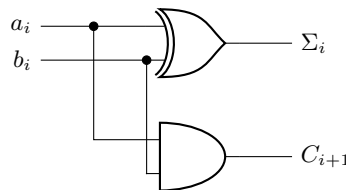


Tabella di verità del semi-sommatore:

a_i	b_i	Σ_i	C_{i+1}
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

3.8.2 Sommatore completo (full adder)

Il **sommatore completo** (full adder) risolve il limite del semi-sommatore aggiungendo un terzo ingresso C_i per il riporto proveniente da uno stadio precedente. L'uscita somma Σ_i vale 1 se il numero di ingressi a 1 è dispari, mentre il riporto C_{i+1} vale 1 se almeno due ingressi sono a 1.

Il circuito può essere ottenuto combinando due semi-sommatori e una porta OR finale per i riporti, in modo da tenere conto di tutti gli addendi:

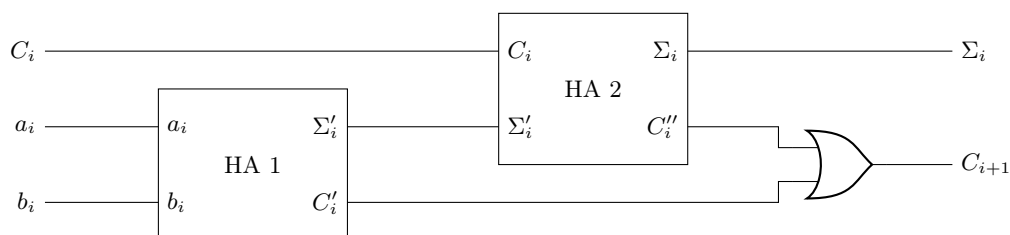


Tabella di verità del sommatore completo (con Σ'_i , C'_{i+1} uscite del primo HA):

a_i	b_i	C_i	Σ'_i	Σ_i	C_{i+1}
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	1	1	0
0	1	1	1	0	1
1	0	0	1	1	0
1	0	1	1	0	1
1	1	0	0	0	1
1	1	1	0	1	1

Logica dietro la tabella di verità

Le due uscite del full adder obbediscono a regole semplici che si ricavano direttamente dal significato fisico della somma binaria.

- **Bit di somma Σ_i** — “**parità**” degli ingressi: $\Sigma_i = 1$ ogni volta che il numero totale di ingressi a **1** è *dispari* (uno solo oppure tutti e tre). Questo coincide esattamente con l’**XOR a tre vie**:

$$\Sigma_i = a_i \oplus b_i \oplus C_i$$

Si può verificare intuitivamente: $1 + 1 + 0 = 2_{10} = 10_2$, la cifra delle unità è **0**; $1 + 1 + 1 = 3_{10} = 11_2$, la cifra delle unità è **1**. In entrambi i casi conta solo la parità degli addendi.

- **Bit di riporto C_{i+1}** — “**maggioranza**” degli ingressi: $C_{i+1} = 1$ ogni volta che *almeno due* ingressi su tre sono a **1** (funzione di maggioranza). Algebricamente:

$$C_{i+1} = a_i b_i + b_i C_i + a_i C_i$$

Ogni termine rappresenta una coppia che, sommata, produce riporto ($1 + 1 \geq 2$).

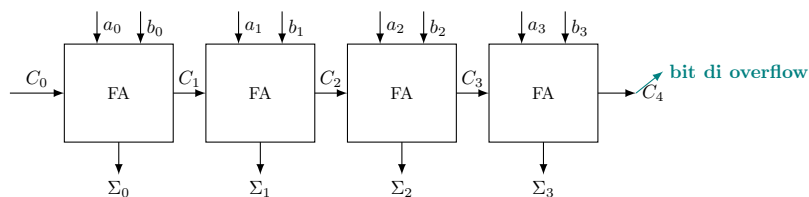
Queste due formule — e quindi l’intera tabella — si ricavano senza dover enumerare tutti gli 8 casi: basta ricordare che Σ_i conta la *parità* e C_{i+1} la *maggioranza* dei tre bit in ingresso.

Capitolo 4. Sommatore, complemento a 2 e codici

4.1 Esempio: sommatore a 4 bit

(27 marzo 2026)

Collegando in cascata 4 sommatore completi (Full Adder, FA) si ottiene un circuito per l'addizione a 4 bit, noto come **Ripple-Carry Adder** (sommatore a propagazione di riporto). Ogni stadio i somma i bit a_i e b_i più il riporto dello stadio precedente (C_i), generando il bit di somma (Σ_i) e il nuovo riporto (C_{i+1}) per lo stadio successivo.



Esempio rapido di calcolo:

Proviamo a sommare $A = 0101_2$ (5_{10}) e $B = 0011_2$ (3_{10}), ponendo il riporto iniziale $C_0 = 0$:

- **Stadio 0:** $a_0(1) + b_0(1) + C_0(0) = 2 \Rightarrow \Sigma_0 = 0$, riporto $C_1 = 1$.
- **Stadio 1:** $a_1(0) + b_1(1) + C_1(1) = 2 \Rightarrow \Sigma_1 = 0$, riporto $C_2 = 1$.
- **Stadio 2:** $a_2(1) + b_2(0) + C_2(1) = 2 \Rightarrow \Sigma_2 = 0$, riporto $C_3 = 1$.
- **Stadio 3:** $a_3(0) + b_3(0) + C_3(1) = 1 \Rightarrow \Sigma_3 = 1$, riporto $C_4 = 0$.

Risultato corretto: $\Sigma = 1000_2$ (8_{10}).

Bit di overflow (C_4): È il bit di riporto finale in uscita dal blocco più significativo. Se stiamo eseguendo addizioni su numeri binari puri (senza segno) a 4 bit e otteniamo $C_4 = 1$, il risultato supera il valore massimo rappresentabile (15_{10}). In questo caso si è verificato un errore di overflow.

Il problema del ritardo (Delay):

Il principale limite prestazionale di questo circuito è che ogni blocco FA deve "aspettare" che il blocco precedente calcoli il suo riporto. Il segnale C deve letteralmente "propagarsi" a cascata (ripple) da C_0 fino a C_4 .

- Se una singola porta logica ha un tempo di ritardo ΔT , ogni blocco Full Adder impiega circa $3\Delta T$ per generare il riporto.
- In un sommatore a n bit, il tempo totale Δt cresce in modo lineare:

$$\Delta t = n \cdot 3\Delta T$$

Soluzione: i sommatore Carry Look-Ahead (CLA)

Per eliminare questa latenza si utilizzano circuiti più avanzati (ad anticipo di riporto). Attraverso un'apposita rete logica parallela, questi circuiti capiscono "in anticipo" se i vari stadi andranno a *generare* o *propagare* un riporto. In questo modo si possono calcolare tutte le uscite simultaneamente, con un ritardo bassissimo e logaritmico (o indipendente) rispetto alla lunghezza n della parola.

4.2 Modularità e Design Gerarchico

Nella progettazione digitale moderna, si sfrutta ampiamente il concetto di **modularità**.

Design Modulare: un approccio in cui il sistema complesso viene suddiviso in sottomoduli standard e intercambiabili (es. usare i Full Adder base per costruire un sommatore a n bit). Questi moduli possono essere aggiunti, rimossi o sostituiti senza dover alterare o riprogettare il resto del sistema.

I vantaggi principali includono:

- **Astrazione:** chi progetta un processore non deve preoccuparsi della logica a porte interne al singolo FA, ma ne considera solo gli ingressi e le uscite.
- **Riutilizzo e Semplificazione:** un modulo testato e verificato può essere riutilizzato in più punti, rendendo il collaudo molto più affidabile.

4.3 Sottrattori e numeri negativi

(27 marzo 2026 – cont.)

Nei circuiti digitali, per rappresentare anche i numeri negativi, si deve "sacrificare" un bit (di solito il più significativo) per indicare il segno. Esistono diverse tecniche per farlo:

1. Modulo e Segno

Il bit più a sinistra indica il segno (0 = positivo, 1 = negativo), mentre i restanti bit indicano il valore assoluto. Questa tecnica presenta un problema pratico: lo zero ha due rappresentazioni diverse. Ad esempio su 4 bit, 0000 (+0) e 1000 (−0) indicano entrambi il numero zero. Questo complica inutilmente i circuiti di calcolo.

2. La teoria dei Complementi

Per risolvere questo problema si utilizza un metodo matematico chiamato *complemento*. Dato un numero N espresso in una certa base r e lungo n cifre, si definiscono due tipi di complemento:

- **Complemento alla base diminuita (base $r - 1$):**
Si definisce come $C_{r-1}(N) = (r^n - 1) - N$. Poiché $r^n - 1$ è il numero massimo rappresentabile ($N_{\max}$), si ottiene sottraendo il numero N dal valore massimo.
- **Complemento alla base (base r):**
Si definisce come $C_r(N) = r^n - N$. Essendo $r^n = (r^n - 1) + 1$, equivale matematicamente ad aggiungere 1 al complemento alla base diminuita.

Applicazione al Sistema Binario (Base 2):

Nei sistemi binari (base $r = 2$), queste due definizioni prendono il nome di **Complemento a 1** e **Complemento a 2**.

Complemento a 1 (C_1):

$$C_1(N) = (2^n - 1) - N$$

Poiché $2^n - 1$ è un numero composto da tutti "1" (es. 1111_2), sottrarre N significa banalmente *invertire tutti i singoli bit* di N (ogni bit N_i diventa $\overline{N}_i$):

$$C_1(N) = \overline{N}_{n-1} \cdots \overline{N}_1 \overline{N}_0$$

Complemento a 2 (C_2):

$$C_2(N) = 2^n - N$$

Come visto prima, possiamo riscriverlo come $(2^n - 1 - N) + 1$. Questo significa che per calcolare il complemento a 2 basta prendere il complemento a 1 e sommare 1:

$$C_2(N) = C_1(N) + 1 = \overline{N}_{n-1} \cdots \overline{N}_1 \overline{N}_0 + 1$$

Proprietà fondamentale: Applicare due volte il complemento a 2 restituisce il numero originale, ovvero $C_2(C_2(N)) = N$.

Il **complemento a 2** è la notazione standard per rappresentare numeri interi con segno nei calcolatori. A differenza della codifica a modulo e segno, garantisce un'unica rappresentazione per lo zero (es. 0000) e permette di eseguire somme e sottrazioni con la stessa rete logica hardware.

Regola pratica per calcolare il complemento a 2 di un numero:

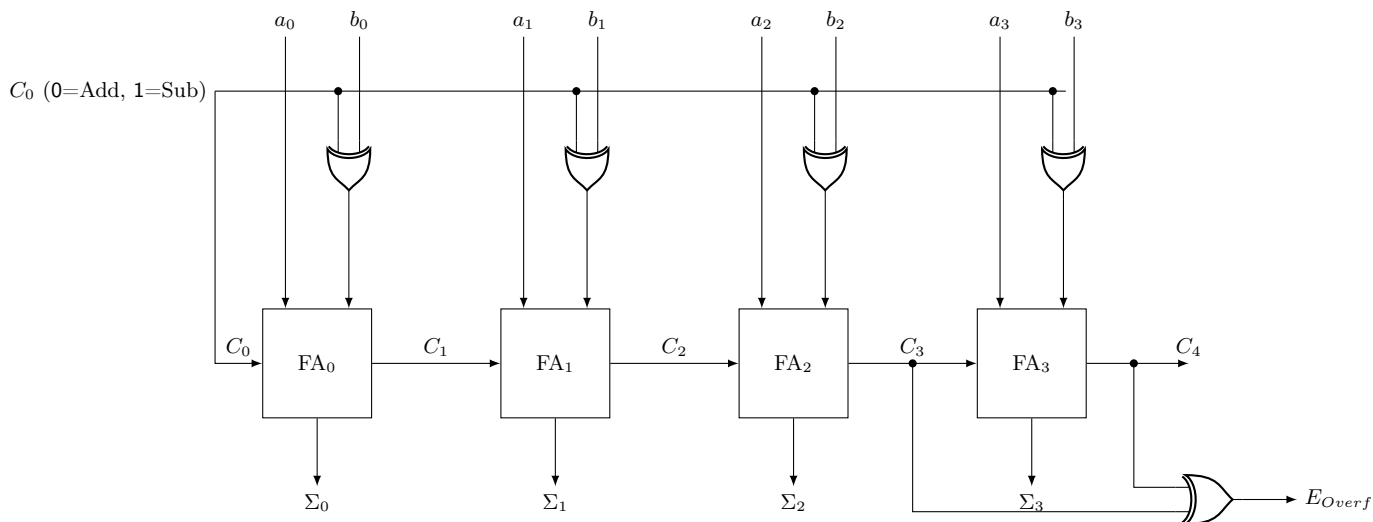
1. Si parte dal numero binario positivo (es. $+3_{10} = 0011_2$).
2. Si invertono tutti i bit (complemento a 1): $0011 \rightarrow 1100$.
3. Si somma 1 al risultato: $1100 + 1 = 1101$.

Quindi $-3_{10} = 1101_2$ in complemento a 2 su 4 bit.

Se verifichiamo calcolando l'addizione $-3 + 3$, avremo: $1101 + 0011 = 10000$. Poiché lavoriamo a 4 bit, l'uno più a sinistra (il *carry out* uscente) cade fuori dalla dimensione dei registri e viene ignorato. Il risultato memorizzato è esattamente 0000, ovvero 0.

4.3.1 Circuito sommatore/sottrattore a 4 bit

Un grande vantaggio della codifica in complemento a 2 è la possibilità di eseguire sia addizioni che sottrazioni utilizzando il **medesimo circuito hardware**. Per farlo, si estende il classico sommatore (ripple-carry adder) introducendo delle porte logiche XOR sugli ingressi del secondo operando e sfruttando il riporto iniziale C_0 come segnale di controllo (selettore d'operazione).

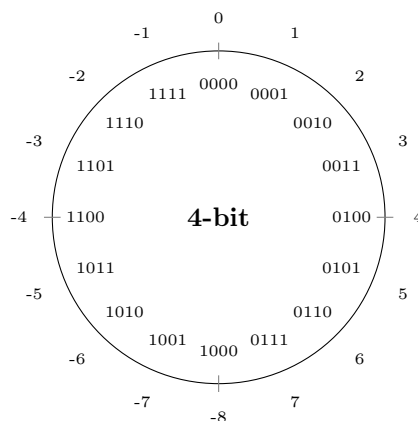


Il funzionamento del circuito dipende dal valore del segnale di controllo C_0 :

- **Se $C_0 = 0$ (Addizione):** la porta XOR riceve uno 0 e si comporta come un buffer trasparente ($b_i \oplus 0 = b_i$). Inoltre, il riporto in ingresso al primo FA è anch'esso 0. Il circuito esegue quindi una normale addizione: $S = a + b$.
- **Se $C_0 = 1$ (Sottrazione):** la porta XOR riceve un 1 e si comporta come un invertitore logico ($b_i \oplus 1 = \bar{b}_i$), calcolando il complemento a 1 di b . Il riporto in ingresso al primo FA fornisce un ulteriore +1. In questo modo, il circuito calcola internamente l'espressione $a + \bar{b} + 1$. Poiché $\bar{b} + 1$ equivale a $-b$ (in complemento a 2), l'operazione totale svolta è esattamente la sottrazione: $S = a - b$.
- **Errore di Overflow (E):** lavorando con un numero fisso di bit (in questo caso 4 bit, intervallo rappresentabile da -8 a $+7$), può accadere che il risultato reale dell'operazione sia al di fuori di questo intervallo. Questo errore prende il nome di *overflow* e viene segnalato tramite una porta XOR applicata agli ultimi due riporti generati ($E = C_3 \oplus C_4$). Se $E = 1$, si ha un *overflow* e il risultato non è valido.

In generale, lavorando in complemento a 2 con 4 bit, si rappresenta l'intervallo da -8 a $+7$:

- Se $\Sigma_3 = 0 \Leftrightarrow$ il risultato Σ è positivo o nullo.
- Se $\Sigma_3 = 1 \Leftrightarrow$ il risultato Σ è negativo.



Capitolo 5. Codice Gray e ASCII

(30 marzo 2026)

5.1 Codice Gray

Il **Codice Gray** è una codifica binaria non pesata in cui due valori consecutivi differiscono sempre e solo per *un singolo bit*.

I vantaggi principali del codice Gray rispetto al normale codice binario puro sono:

- **Prevenzione dei glitch (stati spuri):** in un contatore binario normale, il passaggio ad esempio da 3 (011_2) a 4 (100_2) comporta la commutazione simultanea di 3 bit. Fisicamente, i bit non commuteranno mai nel medesimo istante esatto, causando passaggi in stati intermedi transitori errati (es. $011 \rightarrow 111 \rightarrow 100$). Con il codice Gray, cambiando un solo bit per ogni incremento, le transizioni sono sempre sicure e prive di glitch.
- **Riduzione della dissipazione di potenza:** in tecnologie CMOS, la potenza dinamica è dissipata principalmente durante le commutazioni di stato logico. Poiché nel codice Gray commuta sempre e solo un bit alla volta, la potenza dissipata è minima e costante nel tempo, rendendolo ideale per contatori a basso consumo o per le macchine a stati finiti.

Dec.	Binario	Gray
0	0000	0000
1	0001	0001
2	0010	0011
3	0011	0010
4	0100	0110
5	0101	0111
6	0110	0101
7	0111	0100
8	1000	1100
9	1001	1101
10	1010	1111
11	1011	1110
12	1100	1010
13	1101	1011
14	1110	1001
15	1111	1000

Dato un numero a n bit (dove l'indice $n - 1$ rappresenta il bit più significativo, MSB):

Conversione Binario $\rightarrow$ Gray:

Il bit più significativo resta invariato. Ogni altro bit Gray si calcola come XOR tra il bit binario nella stessa posizione e il bit binario precedente (più significativo).

$$G_{n-1} = B_{n-1} \quad G_k = B_k \oplus B_{k+1} \quad \text{per } k < n - 1$$

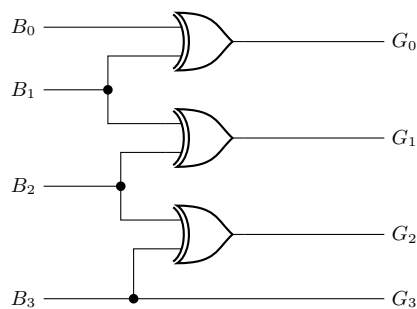
Conversione Gray $\rightarrow$ Binario:

Il bit più significativo resta invariato. Ogni altro bit binario si ottiene come XOR tra il bit Gray nella stessa posizione e il *bit binario appena calcolato* nella posizione più significativa adiacente.

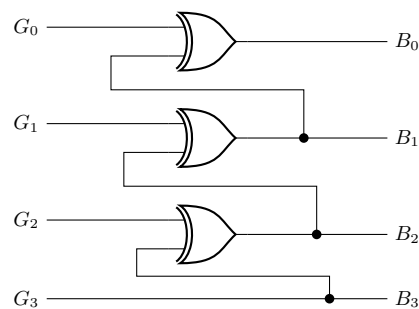
$$B_{n-1} = G_{n-1} \quad B_k = G_k \oplus B_{k+1} \quad \text{per } k < n - 1$$

(Nota matematica: sviluppando la ricorsione si nota che equivale alla formula $B_k = G_k \oplus G_{k+1} \oplus \dots \oplus G_{n-1}$, spesso implementata in hardware per velocizzare i tempi di calcolo parallelo).

5.1.1 Circuiti di conversione



Conversione B $\rightarrow$ G



Conversione G $\rightarrow$ B

5.1.2 Esercizi svolti: conversioni passo-passo

Applichiamo le formule ai due casi classici usando numeri a **4 bit** ($B_3B_2B_1B_0$, con $B_3 = \text{MSB}$).

Esercizio 1 — Binario $\rightarrow$ Gray: $1011_2 (= 11_{10})$

La regola è: $G_3 = B_3$, poi $G_k = B_k \oplus B_{k+1}$.

Bit	Formula	Calcolo	Risultato
G_3	B_3	1	$G_3 = \mathbf{1}$
G_2	$B_2 \oplus B_3$	$0 \oplus 1$	$G_2 = \mathbf{1}$
G_1	$B_1 \oplus B_2$	$1 \oplus 0$	$G_1 = \mathbf{1}$
G_0	$B_0 \oplus B_1$	$1 \oplus 1$	$G_0 = \mathbf{0}$

Si parte dal MSB e si scende verso il LSB, applicando uno XOR bit a bit con il vicino più significativo già noto:

$$\underbrace{1}_{B_3} \underbrace{0}_{B_2} \underbrace{1}_{B_1} \underbrace{1}_{B_0} \xrightarrow{B \rightarrow G} \underbrace{1}_{G_3} \underbrace{1}_{G_2} \underbrace{1}_{G_1} \underbrace{0}_{G_0} \Rightarrow \boxed{1110_G}$$

Esercizio 2 — Gray $\rightarrow$ Binario: 1110_G

La regola è: $B_3 = G_3$, poi $B_k = G_k \oplus B_{k+1}$ (si usa il binario *già calcolato* al passo precedente, non il Gray).

Bit	Formula	Calcolo	Risultato
B_3	G_3	1	$B_3 = \mathbf{1}$
B_2	$G_2 \oplus B_3$	$1 \oplus 1$	$B_2 = \mathbf{0}$
B_1	$G_1 \oplus B_2$	$1 \oplus 0$	$B_1 = \mathbf{1}$
B_0	$G_0 \oplus B_1$	$0 \oplus 1$	$B_0 = \mathbf{1}$

Anche qui si procede dall'MSB verso l'LSB, ma ogni XOR usa il *bit binario appena ricavato* (non il Gray):

$$\underbrace{1}_{G_3} \underbrace{1}_{G_2} \underbrace{1}_{G_1} \underbrace{0}_{G_0} \xrightarrow{G \rightarrow B} \underbrace{1}_{B_3} \underbrace{0}_{B_2} \underbrace{1}_{B_1} \underbrace{1}_{B_0} \Rightarrow \boxed{1011_2 = 11_{10}}$$

I due esercizi si invertono a vicenda: il risultato di Esercizio 2 coincide esattamente con il punto di partenza di Esercizio 1, confermando la correttezza delle due conversioni.

Esercizio 3 — Binario $\rightarrow$ Gray: $1101_2 (= 13_{10})$

Bit	Formula	Calcolo	Risultato
G_3	B_3	1	$G_3 = \mathbf{1}$
G_2	$B_2 \oplus B_3$	$1 \oplus 1$	$G_2 = \mathbf{0}$
G_1	$B_1 \oplus B_2$	$0 \oplus 1$	$G_1 = \mathbf{1}$
G_0	$B_0 \oplus B_1$	$1 \oplus 0$	$G_0 = \mathbf{1}$

$$1101_2 \xrightarrow{B \rightarrow G} \boxed{1011_G}$$

Esercizio 4 — Gray $\rightarrow$ Binario: 1011_G

Bit	Formula	Calcolo	Risultato
B_3	G_3	1	$B_3 = \mathbf{1}$
B_2	$G_2 \oplus B_3$	$0 \oplus 1$	$B_2 = \mathbf{1}$
B_1	$G_1 \oplus B_2$	$1 \oplus 1$	$B_1 = \mathbf{0}$
B_0	$G_0 \oplus B_1$	$1 \oplus 0$	$B_0 = \mathbf{1}$

$$1011_G \xrightarrow{G \rightarrow B} \boxed{1101_2 = 13_{10}}$$

Anche qui la coppia Esercizio 3 e Esercizio 4 è inversa, come atteso.

5.2 Codice ASCII

L'**ASCII** (*American Standard Code for Information Interchange*) è il sistema di codifica standard utilizzato storicamente per rappresentare caratteri alfanumerici e simboli di controllo nei dispositivi digitali e di telecomunicazione.

La tabella ASCII standard originale utilizza **7 bit** per codificare 128 simboli diversi ($2^7 = 128$). I codici sono suddivisi logicamente in:

- **0–31 e 127:** *Caratteri di controllo* (non stampabili), usati originariamente per gestire la formattazione hardware delle telescriventi (es. ritorno a capo **CR**, avanzamento riga **LF**, tabulazione **HT**) o il protocollo di comunicazione.
- **32–47, 58–64, 91–96, 123–126:** *Simboli grafici e punteggiatura* (lo spazio è il primo carattere stampabile, codice decimale 32).
- **48–57:** *Cifre numeriche* (da '0' a '9').
- **65–90:** *Lettere maiuscole* (da 'A' a 'Z').
- **97–122:** *Lettere minuscole* (da 'a' a 'z').

Proprietà notevole: le lettere maiuscole e le rispettive minuscole differiscono sempre esattamente per *un solo bit* (il sesto bit partendo da zero, che vale $2^5 = 32$). Ad esempio, la 'A' vale 65_{10} (01000001_2) e la 'a' vale 97_{10} (01100001_2). Questa ingegnosa scelta progettuale non è casuale: permette ai sistemi digitali e ai microprocessori di convertire istantaneamente e a basso costo computazionale i caratteri dal maiuscolo al minuscolo semplicemente invertendo un bit (ad esempio applicando una maschera logica di tipo XOR o OR).

Dec	Oct	Hex	Char	Dec	Oct	Hex	Char	Dec	Oct	Hex	Char	Dec	Oct	Hex	Char
0	000	00	NUL	32	040	20	(sp)	64	100	40	@	96	140	60	`
1	001	01	SOH	33	041	21	!	65	101	41	A	97	141	61	a
2	002	02	STX	34	042	22	"	66	102	42	B	98	142	62	b
3	003	03	ETX	35	043	23	#	67	103	43	C	99	143	63	c
4	004	04	EOT	36	044	24	\$	68	104	44	D	100	144	64	d
5	005	05	ENQ	37	045	25	%	69	105	45	E	101	145	65	e
6	006	06	ACK	38	046	26	&	70	106	46	F	102	146	66	f
7	007	07	BEL	39	047	27	'	71	107	47	G	103	147	67	g
8	010	08	BS	40	050	28	(	72	110	48	H	104	150	68	h
9	011	09	HT	41	051	29	)	73	111	49	I	105	151	69	i
10	012	0A	LF	42	052	2A	*	74	112	4A	J	106	152	6A	j
11	013	0B	VT	43	053	2B	+	75	113	4B	K	107	153	6B	k
12	014	0C	FF	44	054	2C	,	76	114	4C	L	108	154	6C	l
13	015	0D	CR	45	055	2D	-	77	115	4D	M	109	155	6D	m
14	016	0E	SO	46	056	2E	.	78	116	4E	N	110	156	6E	n
15	017	0F	SI	47	057	2F	/	79	117	4F	O	111	157	6F	o
16	020	10	DLE	48	060	30	0	80	120	50	P	112	160	70	p
17	021	11	DC1	49	061	31	1	81	121	51	Q	113	161	71	q
18	022	12	DC2	50	062	32	2	82	122	52	R	114	162	72	r
19	023	13	DC3	51	063	33	3	83	123	53	S	115	163	73	s
20	024	14	DC4	52	064	34	4	84	124	54	T	116	164	74	t
21	025	15	NAK	53	065	35	5	85	125	55	U	117	165	75	u
22	026	16	SYN	54	066	36	6	86	126	56	V	118	166	76	v
23	027	17	ETB	55	067	37	7	87	127	57	W	119	167	77	w
24	030	18	CAN	56	070	38	8	88	130	58	X	120	170	78	x
25	031	19	EM	57	071	39	9	89	131	59	Y	121	171	79	y
26	032	1A	SUB	58	072	3A	:	90	132	5A	Z	122	172	7A	z
27	033	1B	ESC	59	073	3B	;	91	133	5B	[	123	173	7B	{
28	034	1C	FS	60	074	3C	<	92	134	5C	\	124	174	7C	
29	035	1D	GS	61	075	3D	=	93	135	5D	]	125	175	7D	}
30	036	1E	RS	62	076	3E	>	94	136	5E	^	126	176	7E	~
31	037	1F	US	63	077	3F	?	95	137	5F	_	127	177	7F	DEL

Legenda: le colonne da sinistra a destra mostrano decimale, ottale, esadecimale e il carattere corrispondente. I caratteri di controllo (0–31 e 127) sono abbreviati (es. NUL=null, SOH=start of heading, CR=carriage return, DEL=delete, ecc.).

Intervalli notevoli:

- Caratteri di controllo: 0–31 e 127;
- Spazio: 32;
- Caratteri stampabili: 33–126;
- Cifre: 48–57 (0–9);
- Lettere maiuscole: 65–90 (A–Z);
- Lettere minuscole: 97–122 (a–z).

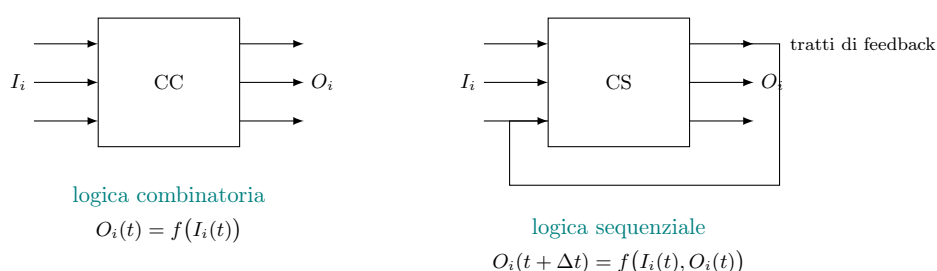
Capitolo 6. Logica sequenziale

6.1 Circuiti combinatori e circuiti sequenziali

Finora abbiamo visto circuiti in cui l'output dipende esclusivamente dagli input: **circuiti combinatori**.

Esistono anche circuiti in cui lo stato delle uscite dipende anche dallo stato in cui il circuito era in precedenza: questi circuiti seguono una **logica sequenziale**.

Nei circuiti sequenziali è necessario che ci sia una **memoria**, un feedback.

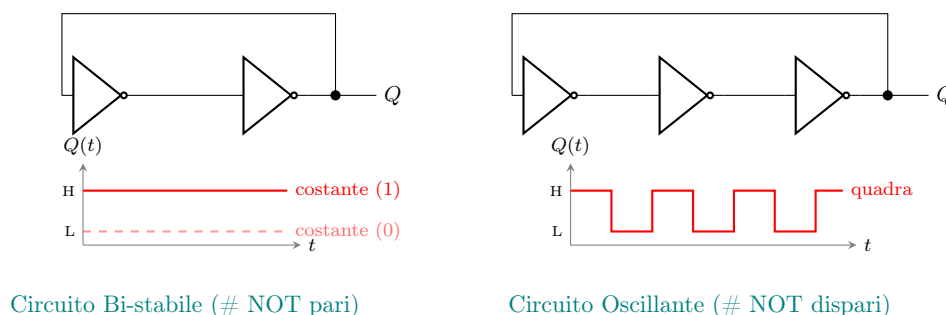


6.2 Bistabilità e oscillazione

Il **circuito bistabile** rappresenta la forma più semplice di **memoria** in elettronica digitale. Si ottiene collegando in anello chiuso un numero **pari** di porte logiche invertenti (ad esempio porte NOT). Poiché ogni segnale viene invertito due volte, il valore che ritorna all'ingresso è uguale a quello di partenza. Questo crea un **feedback positivo** che rinforza e mantiene stabile all'infinito lo stato logico assunto all'accensione, conservando così un bit di informazione (0 o 1).

Il problema di un semplice anello di porte NOT è che è un sistema isolato: **non è possibile modificarne lo stato** dall'esterno. Per potervi "scrivere" un nuovo bit (impostarlo a 1 o azzerarlo a 0), le porte NOT vengono sostituite con porte **NOR** o **NAND**, le quali offrono dei terminali di ingresso aggiuntivi per i segnali di controllo.

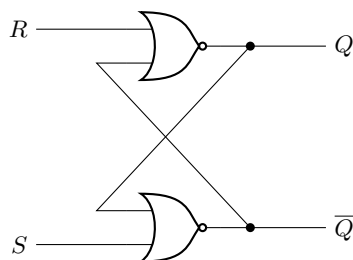
Se invece colleghiamo in anello chiuso un numero **dispari** di porte NOT, otteniamo un **circuito oscillante** (o *ring oscillator*). L'inversione dispari fa sì che il segnale ritorni all'ingresso invertito rispetto alla partenza. Questo genera una perenne instabilità: l'uscita continua a commutare ciclicamente tra 0 e 1, con un periodo T proporzionale al numero di porte e al ritardo di propagazione di ciascuna di esse.



6.3 Latch

Latch = Il circuito di memoria fondamentale dell'elettronica digitale sequenziale, capace di conservare un bit e di essere aggiornato tramite appositi segnali di ingresso.

Il Latch più semplice (Latch SR, *Set-Reset*) nasce dall'evoluzione del circuito bistabile: sostituendo le due porte NOT con due porte **NOR** (o NAND) e incrociando i collegamenti (*cross-coupling*), otteniamo due ingressi di controllo: **S** (*Set*) e **R** (*Reset*).



Latch SR con porte NOR

Funzionamento e Tabella di verità del Latch SR con porte NOR

Le uscite sono Q e $\bar{Q}$. Il pedice n indica lo stato *attuale* del latch (prima che gli ingressi cambino), mentre $n+1$ indica il nuovo stato che il circuito assumerà.

R	S	Q_{n+1}	$\bar{Q}_{n+1}$	Comportamento
0	0	Q_n	$\bar{Q}_n$	MEMORIA (stato mantenuto)
0	1	1	0	SET — forza $Q = 1$
1	0	0	1	RESET — forza $Q = 0$
1	1	0	0	PROIBITO — $Q = \bar{Q} = 0$, uscita incoerente

Di seguito viene spiegato il meccanismo fisico di ogni stato, seguendo il percorso del segnale attraverso le due porte NOR incrociate.

- **Memoria** ($R = 0, S = 0$) Con entrambi gli ingressi a 0, ogni porta NOR ha un solo ingresso attivo: quello proveniente dall'uscita dell'altra porta (*cross-coupling*). Il circuito si comporta esattamente come un anello di due invertitori: qualunque stato ($Q = 0$ o $Q = 1$) è auto-consistente e viene mantenuto indefinitamente. **Non avviene nessun cambiamento**: il bit memorizzato rimane intatto.
- **Set** ($R = 0, S = 1$) L'ingresso $S = 1$ forza l'uscita della porta NOR inferiore a 0 ($\bar{Q} \rightarrow 0$), indipendentemente dal suo secondo ingresso di feedback. Questo 0 si propaga alla porta NOR superiore, la quale ha ora *entrambi* gli ingressi a 0 ($R = 0$ e il feedback $\bar{Q} = 0$), quindi porta la sua uscita a $Q \rightarrow 1$. Il latch si **stabilizza** nello stato $Q = 1, \bar{Q} = 0$.
- **Reset** ($R = 1, S = 0$) Simmetrico al caso precedente: $R = 1$ forza l'uscita della porta NOR superiore a 0 ($Q \rightarrow 0$). Questo 0 raggiunge la porta NOR inferiore che, con entrambi gli ingressi a 0 ($S = 0$ e il feedback $Q = 0$), porta $\bar{Q} \rightarrow 1$. Il latch si **stabilizza** nello stato $Q = 0, \bar{Q} = 1$.
- **Stato proibito** ($R = 1, S = 1$) Con entrambi gli ingressi a 1, *entrambe* le porte NOR vengono forzate a 0: $Q = 0$ e $\bar{Q} = 0$. Questo viola il requisito fondamentale del latch, ovvero che le uscite siano sempre complementari. Ancora più grave è ciò che accade se si riportano poi R e S a 0 simultaneamente: entrambe le porte tenderanno di portare la propria uscita a 1 nello stesso istante. Il risultato finale dipenderà da infinitesime differenze nei ritardi di propagazione e nelle caratteristiche costruttive dei transistor, rendendo l'uscita **imprevedibile** e potenzialmente causando una *metastabilità* o un'oscillazione transitoria. Per questo motivo la combinazione $R = S = 1$ è da evitare sempre.

6.3.1 Latch RS con NOR: funzione caratteristica

In questa sezione ricaviamo *algebricamente* l'equazione che governa il latch con porte NOR. Il procedimento si basa sull'analisi del **circuito equivalente**, in cui l'anello di retroazione viene idealmente interrotto per studiare la propagazione dei segnali da un istante t all'istante successivo $t + \Delta t$.

1. Costruzione dell'equazione dal circuito equivalente. Seguendo il percorso dei segnali nel circuito equivalente:

- La prima porta NOR riceve come ingressi il segnale S e lo stato attuale $Q(t)$ proveniente dalla retroazione. Il suo risultato è l'uscita complementata al tempo t , ovvero $\bar{Q}(t)$:

$$\bar{Q}(t) = \overline{S + Q(t)}$$

- La seconda porta NOR riceve l'ingresso R e il segnale appena calcolato $\overline{Q}(t)$. Il suo risultato è il nuovo stato del latch, $Q(t + \Delta t)$:

$$Q(t + \Delta t) = R + \overline{Q}(t)$$

Sostituendo la prima espressione nella seconda, si ottiene l'equazione di partenza completa:

$$Q(t + \Delta t) = R + \overline{(S + Q(t))}$$

2. Semplificazione con la variabile ausiliaria A . Per semplificare la derivazione, introduciamo una sostituzione di variabile. Chiamiamo A l'espressione all'interno della negazione più interna:

$$A = S + Q(t) \implies \overline{A} = \overline{S + Q(t)}$$

Riscrivendo l'equazione principale con questa sostituzione otteniamo:

$$Q(t + \Delta t) = R + \overline{A}$$

Ora applichiamo il teorema di De Morgan ($\overline{X + Y} = \overline{X} \cdot \overline{Y}$), trasformando la negazione di una somma logica in un prodotto logico di negazioni:

$$Q(t + \Delta t) = R \cdot \overline{A} = R \cdot A$$

3. Espansione e utilizzo dello stato inutilizzato. Tornando all'espressione estesa (sostituendo nuovamente $A = S + Q(t)$), ricaviamo:

$$Q(t + \Delta t) = R \cdot (S + Q(t)) = RS + RQ(t)$$

Introduciamo un termine aggiuntivo sfruttando il fatto che la combinazione d'ingresso $R = 1$ e $S = 1$ rappresenta uno stato non utilizzato (stato proibito). Dalla tabella di verità sappiamo infatti che tale condizione non deve mai verificarsi durante il normale funzionamento, il che implica che il prodotto logico dei due ingressi sia nullo:

$$RS = 0$$

Per le leggi dell'algebra booleana, sommare 0 a un'espressione non ne altera il valore ($X + 0 = X$). Possiamo quindi aggiungere il termine RS all'equazione per facilitare il successivo raccoglimento:

$$Q(t + \Delta t) = \overline{R}S + RQ(t) + RS$$

4. Raccoglimento e formula finale. Riordinando i termini e raccogliendo a fattor comune la variabile S tra il primo e il terzo addendo:

$$Q(t + \Delta t) = S(\overline{R} + R) + RQ(t)$$

Poiché la somma logica di una variabile col suo complemento vale sempre 1 ($\overline{R} + R = 1$), l'espressione si riduce a:

$$Q(t + \Delta t) = S \cdot (1) + RQ(t)$$

Si ricava in questo modo la **funzione caratteristica del latch RS**:

$$Q(t + \Delta t) = S + \overline{R}Q(t)$$

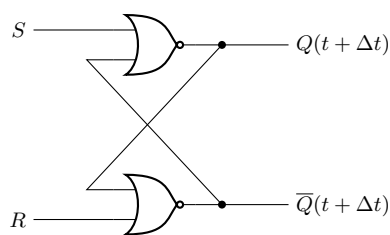
Lettura intuitiva della formula: il nuovo stato $Q(t + \Delta t)$ si porta a 1 se viene richiesto un Set ($S = 1$), oppure se non viene richiesto un Reset ($\overline{R} = 1$) e il circuito si trovava già nello stato 1 ($Q(t) = 1$, mantenimento in memoria).

Verifica della formula sulle quattro combinazioni:

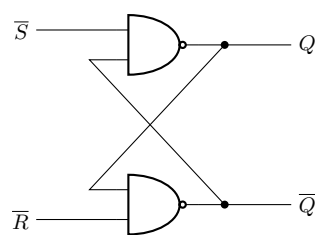
R	S	Formula $S + \overline{R}Q(t)$	$Q(t + \Delta t)$	$\overline{Q}(t + \Delta t)$	Stato
0	0	$0 + 1 \cdot Q(t) = Q(t)$	$Q(t)$	$\overline{Q}(t)$	HOLD
0	1	$1 + 1 \cdot Q(t) = 1$	1	0	SET
1	0	$0 + 0 \cdot Q(t) = 0$	0	1	RESET
1	1	$1 + 0 \cdot Q(t) = 1$ *	0 *	0 *	PROIBITO

* Con $R = S = 1$ la formula darebbe $Q = 1$, ma fisicamente il circuito forza *entrambe* le uscite a 0 (la condizione $R = 1$ prevale sulla porta NOR superiore): è un'incoerenza che conferma il divieto d'uso di questa combinazione.

Schema circuitale e variante NAND. I due schemi seguenti mostrano il latch con porte NOR (a sinistra) e la sua variante con porte NAND (a destra).



Latch SR con porte NOR



Latch SR con porte NAND

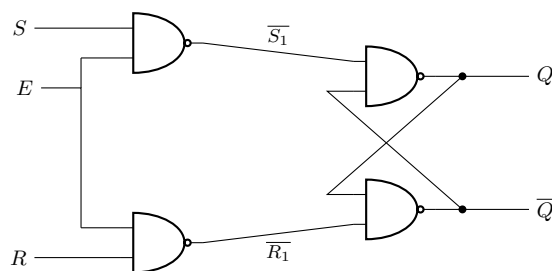
Differenza tra versione NOR e versione NAND. Nel latch con porte **NAND** gli ingressi attivi sono *bassi* anziché alti: $\bar{S} = 0$ attiva il Set e $\bar{R} = 0$ attiva il Reset (logica *active-low*). La funzione caratteristica ha la stessa forma, ma riscritta con gli ingressi complementati:

$$Q(t + \Delta t) = \bar{S} + RQ(t) \iff (\text{in termini di } S \text{ e } R): Q(t + \Delta t) = S + \bar{R}Q(t)$$

Lo **stato proibito** per la versione NAND è $\bar{R} = \bar{S} = 0$ (cioè $R = S = 1$ in logica active-high), per gli stessi motivi visti sopra.

6.3.2 Latch RS con abilitazione (Enable)

I circuiti visti finora sono sempre attivi: ogni variazione degli ingressi si ripercuote immediatamente sulle uscite. Per poter controllare *quando* il circuito deve accettare nuovi dati, si introduce un segnale di abilitazione E (Enable) aggiungendo due porte NAND in ingresso:

Latch di tipo RS (con abilitazione E)

Il comportamento del circuito dipende dal livello del segnale di abilitazione:

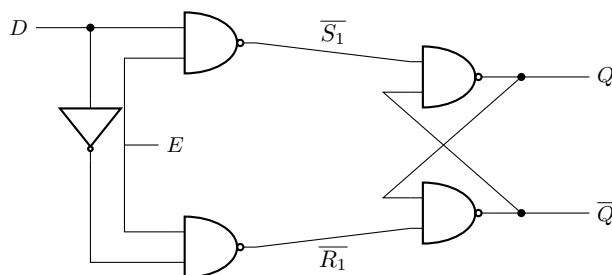
- **Se $E = 0$ (HOLD):** Le due porte NAND di ingresso sono forzate a dare 1 in uscita, quindi $\bar{S}_1 = \bar{R}_1 = 1$. Per il Latch NAND a valle, ricevere un doppio 1 in ingresso significa "mantieni lo stato". Qualsiasi cambiamento sui pin R o S viene bloccato e ignorato: il circuito è **isolato** e conserva la memoria.
- **Se $E = 1$ (TRASPARENTE):** Le porte NAND di ingresso si comportano come semplici invertitori per i segnali S e R . Il circuito funziona esattamente come il normale Latch SR visto in precedenza. Purtroppo, essendo attivo, rimane il difetto dello **stato proibito** se $R = S = 1$.

Funzione caratteristica del latch RS con abilitazione:

$$Q(t + \Delta t) = ES + \bar{R}\bar{E}Q(t)$$

6.3.3 Latch di tipo D

Per evitare definitivamente il rischio di finire nello stato proibito ($R = 1, S = 1$), si vincolano i due ingressi in modo che siano sempre l'uno il complemento dell'altro. Inserendo un **invertitore** tra il ramo superiore e quello inferiore, si ottiene un unico ingresso chiamato D (Data). Così facendo, la combinazione proibita è fisicamente impossibile.



Il funzionamento è estremamente semplice ed efficace:

- Se $E = 1$: l'uscita Q **copia esattamente** l'ingresso D (il circuito è detto "trasparente").
- Se $E = 0$: il circuito si isola e **mantiene in memoria** l'ultimo valore che D possedeva un attimo prima che E andasse a zero.

Funzione di trasferimento del latch D:

$$Q(t + \Delta t) = DE + \bar{E}Q(t)$$

(Nota: avendo vincolato $R = \bar{D}$ e $S = D$, l'equazione si è semplificata. La lettura è letterale: se E è attivo prelievo il Dato D , altrimenti mantengo in memoria $Q(t)$).

6.3.4 Latch e Flip-Flop: differenza fondamentale

Per progettare sistemi digitali sequenziali in modo robusto, è cruciale non confondere mai questi due dispositivi:

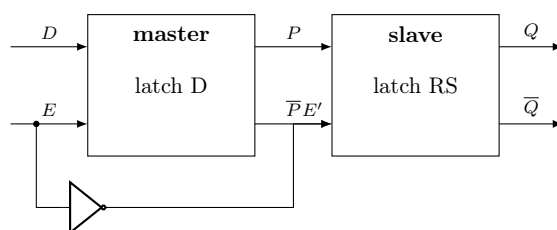
- **LATCH (sensibile a livello)**: Si comporta come una **porta**. Finché il segnale di abilitazione è alto (es. $E = 1$), la porta è "aperta" e i cambiamenti dell'ingresso attraversano liberamente il circuito fino in uscita (comportamento trasparente). Quando l'Enable si abbassa ($E = 0$), la porta si chiude isolando l'uscita, che memorizza così l'ultimo dato transitato.
- **FLIP-FLOP (sensibile a fronte)**: Si comporta come una **macchina fotografica**. L'ingresso viene campionato e trasferito in uscita *esclusivamente* in quell'istante infinitesimo in cui il segnale di controllo (Clock) effettua la transizione, ad esempio da 0 a 1 (fronte di salita). Per tutto il resto del tempo, sia se il clock è stabilmente alto che basso, l'uscita rimane "congelata" sul fotogramma memorizzato e ignora l'ingresso.

Convenzione simbolica: Negli schemi circuitali, se l'ingresso di controllo non riporta segni particolari, si tratta di un Latch (sensibile al livello). Se invece l'ingresso riporta un triangolino ($\triangleright$), si tratta di un Flip-Flop (sensibile al fronte).

6.3.5 Tecnica master-slave

Per convertire un circuito sensibile al livello (Latch) in uno sensibile al fronte (Flip-Flop), un approccio classico è la struttura **Master-Slave**. Essa consiste nel mettere in cascata due Latch comandati da segnali di clock opposti tra loro.

Il primo Latch (il **master**) è pilotato dal segnale di clock originario E , mentre il secondo (lo **slave**) riceve la sua versione negata $E' = \bar{E}$ tramite un invertitore. Le variabili intermedie di collegamento prendono il nome di P e $\bar{P}$.

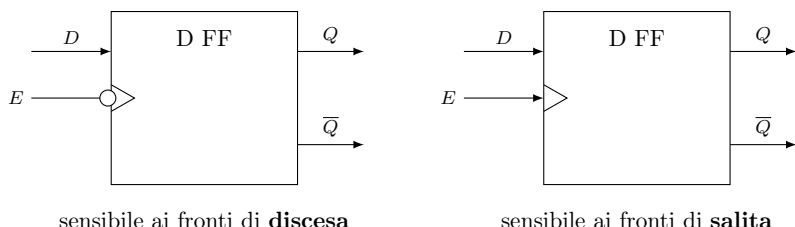


Dinamica di funzionamento del sistema master-slave: Il segreto di questo sistema risiede nel non far mai incontrare ingresso ed uscita in modo diretto. I due latch lavorano a fasi alternate:

- **Fase di Caricamento ($E=1$)**: Il master è trasparente e aggiorna continuamente il proprio stato P copiando l'ingresso D . Lo slave, invece, avendo $E' = 0$, è rigorosamente disabilitato e **congela l'uscita** globale Q allo stato precedente.

- **Transizione e Memorizzazione (E scende a 0):** Nell'istante esatto del fronte di discesa, il master si disabilita catturando definitivamente l'ultimo valore di D sul nodo P . Immediatamente dopo, lo slave si risveglia (E' sale a 1) copiando il valore appena congelato da P sull'uscita finale Q .
- **Fase di Isolamento ($E=0$):** Il master è ormai opaco alle variazioni di D . Lo slave, pur essendo trasparente rispetto a P , non fa altro che riconfermare il dato Q , dato che P non può più cambiare.

Come risultato macroscopico, il passaggio da D a Q si realizza **esclusivamente nel brevissimo istante in cui E si spegne**. Abbiamo ottenuto a tutti gli effetti un Flip-Flop sensibile al fronte di discesa (falling-edge). I due simboli equivalenti del flip-flop master-slave, differenziati per il fronte su cui lavorano attivamente:



Cosa succede se togliamo l'invertitore (NOT)?

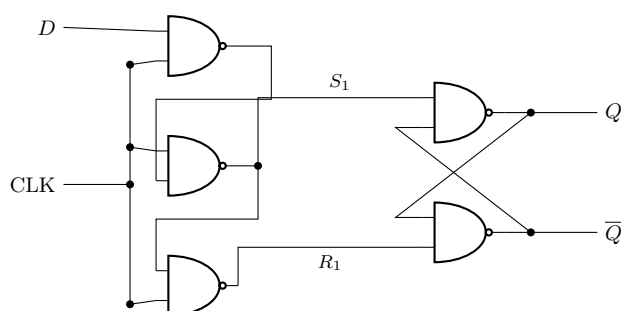
Senza l'inverter, sia il master che lo slave riceverebbero lo stesso segnale E contemporaneamente, vanificando tutto:

- Quando $E = 1$, entrambi i Latch diventerebbero trasparenti nello stesso istante. Il segnale D fluirebbe senza ostacoli attraverso il master, passerebbe per P e attraverserebbe immediatamente lo slave arrivando a Q . L'intero circuito agirebbe come un banale Latch D trasparente, perdendo del tutto la preziosa sensibilità al fronte.
- Quando $E = 0$, entrambi i Latch si chiuderebbero e isolerebbero simultaneamente.

L'inverter è quindi il vero *segreto* del sistema: garantendo che i due Latch non siano **mai trasparenti contemporaneamente**, li forza a passarsi il dato come in una staffetta, permettendo al segnale di raggiungere l'uscita solo nell'esatto istante della commutazione.

6.4 Tecnica trigger-edge

La **tecnica trigger-edge** (sensibile al fronte) rappresenta l'architettura standard per i moderni Flip-Flop integrati. Al posto dell'approccio master-slave (che usa due stadi in cascata), questa tecnica impiega una singola rete fortemente interconnessa, tipicamente formata da cinque porte NAND (o NOR).



Flip-Flop D trigger-edge (5 porte NAND)

Il segreto di questo circuito sta nell'autobloccaggio delle porte:

- Solo durante la brevissima transizione del **fronte attivo** (in questo schema, il fronte di discesa $1 \rightarrow 0$ del CLK) il percorso dati si apre, permettendo a D di pilotare il latch di uscita (Q e $\bar{Q}$).
- Per tutto il resto del tempo, sia quando il CLK è stabilmente a 0 sia quando è stabilmente a 1, la complessa rete di feedback interni forza gli ingressi del latch finale (S_1, R_1) allo stato di riposo, mantenendo l'uscita in memoria e ignorando totalmente le variazioni di D .

Riferimento laboratorio: pag. 9–10–11–12 di [digitale/lab3-09-2026o410-latchFF.pdf](#).

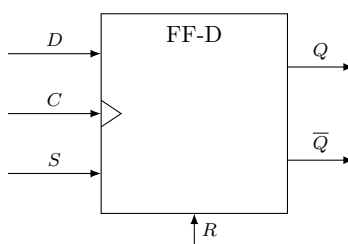
6.5 Ingressi sincroni e asincroni

(13/04/2026)

In un sistema sequenziale pratico, è quasi sempre necessario poter forzare un dispositivo in uno stato noto (es. azzerarlo all'accensione del sistema) senza dover aspettare il battito del clock. Per questo motivo, molti Flip-Flop integrano due tipologie di ingressi contemporaneamente:

- **Ingressi SINCRONI (es. Data D , Clock C):** obbediscono rigidamente alla tempistica del circuito. Un ingresso sincrono (D) viene letto ed applicato alle uscite *esclusivamente* durante il fronte attivo del clock.
- **Ingressi ASINCRONI (es. Preset/Set, Clear/Reset):** agiscono come **comandi di emergenza** o priorità assoluta. Appena vengono attivati, scavalcano totalmente la logica sincrona forzando le uscite istantaneamente, a prescindere da cosa stiano facendo il clock e l'ingresso D .

Esempio di un Flip-Flop D commerciale con ingressi misti (tipico delle logiche TTL):



FF-D con ingressi misti

In questo simbolo, D viene campionato solo sul fronte di C , mentre l'attivazione di S (Set) o R (Reset) forzerà immediatamente l'uscita indipendentemente dal clock.

6.6 JK Flip-Flop

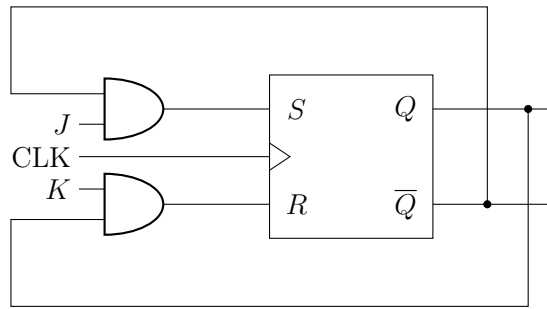
Il Flip-Flop (o Latch) RS originale ha un grave difetto: lo stato $R = 1, S = 1$ è proibito, poiché forza le uscite in uno stato incoerente. Il Flip-Flop di tipo D risolve il problema unendo gli ingressi tramite un invertitore, ma così facendo sacrifica la flessibilità di poter comandare il Set e il Reset in modo indipendente.

Il **Flip-Flop JK** risolve il problema alla radice, mantenendo due ingressi separati (J per il Set e K per il Reset) e introducendo un **incrocio (feedback) dalle uscite verso gli ingressi**. Questo si ottiene antepponendo due porte AND al nucleo RS centrale.

Il principio dell'auto-disabilitazione. Il trucco logico del Flip-Flop JK sta nell'usare le uscite stesse per "spegnere" i comandi ridondanti:

- Se il circuito si trova nello stato $Q = 1$ (e quindi $\bar{Q} = 0$), il segnale $\bar{Q}$ viene retroazionato sulla porta AND dell'ingresso J (Set). Essendo a 0, disabilita completamente l'ingresso J . Il circuito sa di essere già a 1, quindi ignora ulteriori richieste di Set. Allo stesso tempo, l'uscita $Q = 1$ abilita la porta AND dell'ingresso K (Reset).
- Se il circuito è nello stato $Q = 0$ (e $\bar{Q} = 1$), accade l'opposto: viene abilitato l'ingresso J (Set) e viene disabilitato l'ingresso K (Reset).

Grazie a questa intelligenza intrinseca, se applichiamo la combinazione massima $J = 1, K = 1$, il sistema lascerà passare *solo* il comando opposto allo stato in cui si trova, trasformando la vecchia condizione "proibita" in una utilissima funzione di commutazione (Toggle).



JK Flip-Flop

Ricavo della Funzione Caratteristica. Dal circuito logico osserviamo che gli ingressi interni del nucleo RS diventano:

$$S = J \cdot \overline{Q}_n \quad \text{e} \quad R = K \cdot Q_n$$

Partendo dall'equazione caratteristica del Flip-Flop RS, ovvero $Q_{n+1} = S + \overline{R}Q_n$, possiamo sostituire queste due espressioni:

$$\begin{aligned} Q_{n+1} &= (J\overline{Q}_n) + (\overline{KQ}_n)Q_n \\ &= J\overline{Q}_n + (\overline{K} + \overline{Q}_n)Q_n \quad [\text{applicando il Teorema di De Morgan}] \\ &= J\overline{Q}_n + \overline{K}Q_n + \underbrace{\overline{Q}_nQ_n}_{=0} \quad [\text{espandendo e applicando l'annullamento}] \end{aligned}$$

Otteniamo così la **Funzione caratteristica del Flip-Flop JK**:

$$Q_{n+1} = J\overline{Q}_n + \overline{K}Q_n$$

Tabella di verità e sintesi del comportamento:

- $J = 0, K = 0$ (**HOLD / MEMORIA**): Entrambe le porte AND d'ingresso sono bloccate (zero in uscita). Il nucleo centrale riceve la combinazione $S = 0, R = 0$ e quindi **mantiene in memoria** lo stato precedente ($Q_{n+1} = Q_n$).
- $J = 1, K = 0$ (**SET**): L'ingresso J è attivo. Se il flip-flop era a 0, viene portato a 1. Se era già a 1, vi rimane ($Q_{n+1} = 1$).
- $J = 0, K = 1$ (**RESET**): L'ingresso K è attivo. L'uscita viene forzatamente portata a 0, o vi rimane se lo era già ($Q_{n+1} = 0$).
- $J = 1, K = 1$ (**TOGGLE / COMMUTAZIONE**): Il vero punto di forza del JK. Essendo la combinazione proibita ormai innocua, l'applicazione simultanea di due '1' logici forza il sistema a **invertire il proprio stato** ad ogni fronte di clock ($Q_{n+1} = \overline{Q}_n$).

6.6.1 Esempio pratico: il Registro come "filtro anti-glitch"

Per comprendere l'utilità pratica dei circuiti sequenziali, analizziamo un classico problema dei **circuiti combinatori**: la generazione di *glitch* (stati transitori spuri). Immaginiamo di voler pilotare un **display a 7 segmenti** partendo da un dato in codice Gray, utilizzando un **convertitore Gray-Binario**.

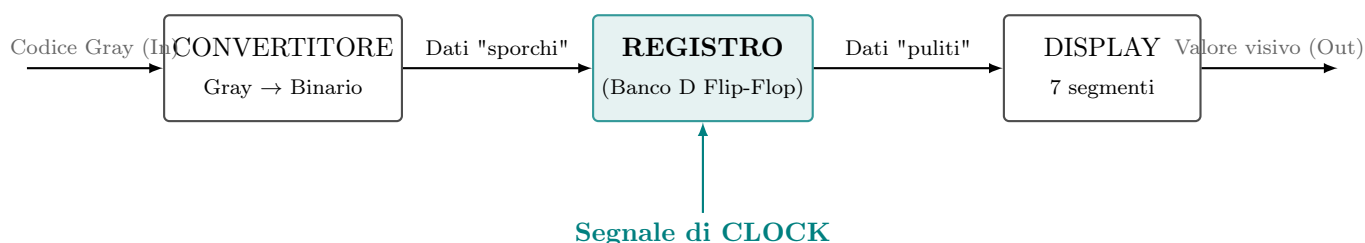
Consideriamo una transizione critica, come il passaggio dal valore 15 al valore 0:

- L'ingresso in codice Gray passa da 1000 (15) a 0000 (0). Come previsto dalla codifica Gray, commuta **un solo bit**.
- L'uscita in codice Binario, invece, deve passare da 1111 a 0000: ben **4 bit devono commutare simultaneamente**.

Il problema: i ritardi di propagazione (Glitch)

Il convertitore è realizzato tramite porte logiche in cascata (tipicamente porte XOR). Poiché ogni porta possiede un proprio **ritardo fisico di propagazione**, i 4 bit in uscita non commuteranno mai nell'esatto medesimo istante. I segnali attraversano percorsi logici di lunghezza diversa, accumulando ritardi differenti.

Di conseguenza, anziché avere un salto netto 1111 → 0000, le uscite attraversano una serie caotica di stati intermedi (ad esempio, 1111 → 0111 → 0011 → 0000). Se il display fosse collegato direttamente all'uscita del convertitore, l'osservatore vedrebbe una rapida e fastidiosa sequenza di numeri errati (sfarfallio) prima della stabilizzazione sul valore corretto. Questi stati transitori indesiderati prendono il nome di *glitch*.

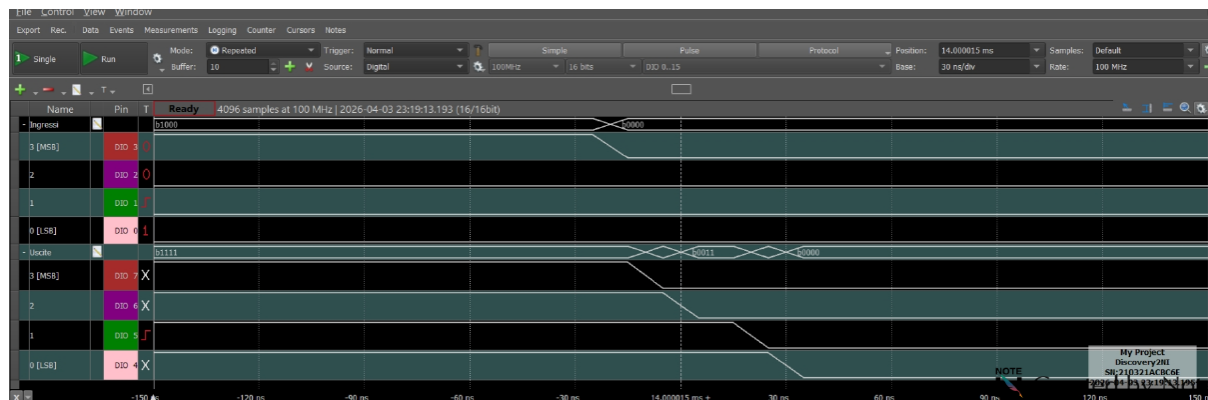


La soluzione: il Registro come barriera di temporizzazione

Per risolvere il problema alla radice, si interpone un **Registro** (un banco di Flip-Flop di tipo D in parallelo, comandati da un unico clock) tra il circuito combinatorio e il display. Il registro agisce come una "fotocamera" sincronizzata, mascherando i transienti:

1. **Fase di assestamento (Glitch):** Mentre il convertitore ricalcola il dato, generando gli stati spuri, il segnale di Clock è mantenuto a riposo. I Flip-Flop ignorano i cambiamenti in ingresso, e il display continua a mostrare stabilmente il numero precedente (15).
2. **Stato stazionario:** Si attende un tempo sufficiente affinché tutte le porte del convertitore abbiano finito di commutare. Ora l'uscita è un 0000 stabile e definitivo.
3. **Campionamento (Fronte attivo):** Proprio in questo istante di stabilità ("acque calme"), si invia un fronte di salita al Clock del registro. I Flip-Flop catturano simultaneamente il nuovo dato pulito e lo trasferiscono al display in blocco.

La traccia seguente, acquisita con un analizzatore logico (Digilent Analog Discovery 2), mostra l'efficacia di questa tecnica:



Osservando il transitorio (a cavallo di $t = 14,000015\text{ ms}$), è evidente come le singole linee in uscita dal blocco combinatorio scendano a zero in istanti sfalsati, generando temporaneamente un dato spuro (es. b0011). Grazie all'azione di campionamento del registro, tali irregolarità vengono confinate a monte: il circuito a valle viene aggiornato con precisione chirurgica e perfetta sincronia solo in corrispondenza del fronte di clock, garantendo un'uscita netta e priva di sfarfallii.

Capitolo 7. Registri a scorrimento e contatori

(17 aprile 2026)

7.1 Registri a scorrimento

Un **registro a scorrimento** (shift register) è una catena di Flip-Flop di tipo D collegati in serie: l'uscita Q di ciascuno è collegata all'ingresso D del successivo. Ad ogni fronte di clock, ogni Flip-Flop «passa» il proprio valore al vicino di destra, come in un nastro trasportatore. In questo modo il dato si *sposta* di una posizione ad ogni colpo di clock.

Per chiarire con un esempio: se il registro contiene i bit $[X, Y, Z]$ e arriva un nuovo dato W dall'ingresso, dopo un tick di clock il contenuto diventa $[W, X, Y]$: il vecchio Z è uscito, X e Y sono slittati di una posizione, e il nuovo W è entrato in testa.

7.1.1 Modalità di trasmissione dati

A seconda di come si alimentano i dati in ingresso e di come si leggono in uscita, lo stesso registro fisico può funzionare in quattro modalità distinte:

- **SISO** (Serial In – Serial Out): il dato entra un bit alla volta dall'ingresso seriale ed esce un bit alla volta dall'ultimo FF. Utilizzo principale: **ritardare** un segnale digitale di esattamente n cicli di clock.
- **SIPO** (Serial In – Parallel Out): il dato entra un bit alla volta, ma si leggono in uscita *contemporaneamente* tutti i bit memorizzati. Utilizzo tipico: convertire una trasmissione seriale (es. linea RS-232) in un dato parallelo leggibile dalla CPU.
- **PISO** (Parallel In – Serial Out): si caricano tutti i bit in un solo colpo (caricamento parallelo), poi li si trasmette uno alla volta. Utilizzo: serializzare un dato parallelo per inviarlo su un bus seriale.
- **PIPO** (Parallel In – Parallel Out): caricamento parallelo e lettura parallela. Funziona come un **registro di memoria**: cattura uno snapshot di un dato a n bit e lo mantiene in uscita fino al prossimo aggiornamento.

7.1.2 Temporizzazione e vincoli

Affinché il registro funzioni correttamente, il clock deve rispettare due vincoli temporali critici propri di ogni Flip-Flop:

$T_P > T_H, \quad T_S + T_P < T_{CLK}$
• T_{setup} (T_S): il dato deve essere stabile sull'ingresso D per almeno questo tempo <i>prima</i> del fronte di clock, altrimenti non viene catturato correttamente. • T_{hold} (T_H): il dato deve rimanere stabile per almeno questo tempo <i>dopo</i> il fronte di clock. • T_P: tempo di propagazione attraverso il FF (quanto ci vuole affinché, dopo il fronte, l'uscita Q si aggiorni e il nuovo valore sia pronto per il FF successivo). • $I_N = D_{SR}$: dato seriale in ingresso al registro.

In parole semplici: il periodo del clock deve essere abbastanza lungo da permettere al segnale di attraversare il FF e di stabilizzarsi sul D del successivo *prima* che arrivi il fronte successivo. Se il clock è troppo veloce si leggono valori errati.

Nei registri a scorrimento sono presenti i seguenti segnali di controllo:

- **CLK**: sincronizza ogni scorrimento.
- **RESET/CLEAR**: azzerava immediatamente l'intero registro.
- **SET**: forza l'uscita a 1 (preset).

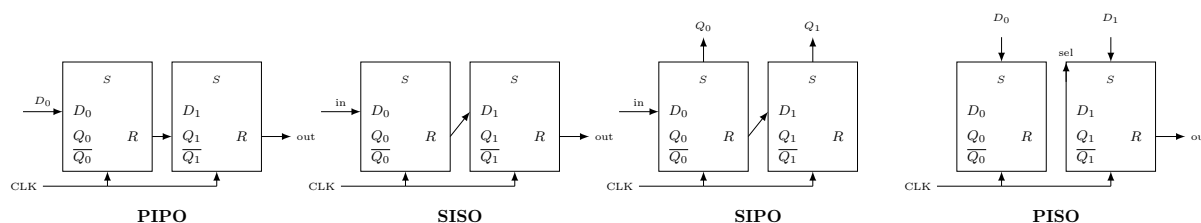
- **ENABLE**: quando basso, congela il registro bloccando gli scorrimenti.

Riassunto – Differenza tra registro normale e registro a scorrimento:

- **Registro normale (PIPO)**: cattura e conserva un dato multi-bit aggiornandolo in blocco. Non c'è scorrimento interno.
- **Registro a scorrimento**: conserva il dato e lo fa scorrere di una posizione ad ogni clock, abilitando la conversione seriale↔parallelo e il ritardo digitale.

7.1.3 Schemi dei registri a scorrimento (SISO, SIPO, PISO)

Di seguito sono riportati gli schemi a blocchi semplificati per le tre configurazioni più comuni (ogni blocco è un FF-D con uscite Q_0 , $\overline{Q_0}$, ingresso D e clock R/CLK).



7.2 Contatori

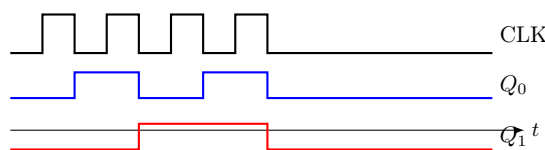
(17 aprile 2026)

Contatore = circuito sequenziale che avanza automaticamente di stato ad ogni fronte di clock, rappresentando in uscita la sequenza binaria 0, 1, 2, 3, ... (o la sua inversa, nei contatori *down*).

L'idea alla base è semplice: ogni bit dell'uscita binaria cambia a frequenza dimezzata rispetto al bit precedente.

- Q_0 (bit meno significativo) commuta ad **ogni** periodo di clock: è il più veloce.
- Q_1 commuta ogni **2** periodi: lo fa quando Q_0 scende da 1 a 0 (fronte di discesa).
- Q_{i+1} commuta ogni 2^{i+1} periodi: in generale ogni bit commuta quando il bit precedente ha un fronte di discesa.

Questo comportamento rende i contatori anche utili come **divisori di frequenza**: il clock ha frequenza f ; Q_0 oscilla a $f/2$; Q_1 a $f/4$; e così via.



7.2.1 Contatore asincrono (ripple counter)

Nel contatore asincrono, il clock **non arriva a tutti i Flip-Flop contemporaneamente**. Solo il primo FF riceve il clock esterno; ogni FF successivo usa come proprio clock l'uscita $\overline{Q}$ del FF precedente. In questo modo, ogni FF scatta automaticamente quando il precedente ha un fronte di discesa.

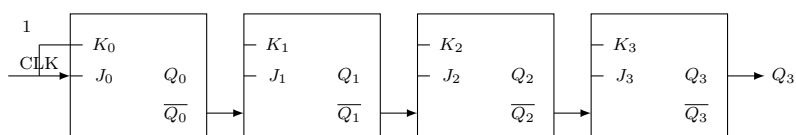
Il problema: la propagazione a catena (ripple). Poiché ogni FF deve aspettare che quello precedente abbia commutato, il segnale si propaga a cascata (da qui il nome *ripple*). Il bit più significativo Q_{n-1} non è aggiornato finché il segnale non ha attraversato tutti gli n FF. Il ritardo totale vale:

$$T_n = n \cdot T_P$$

Questo impone che il periodo del clock sia sufficientemente lungo da permettere a tutta la catena di assestarsi prima del fronte successivo:

$$T_{CLK} > n \cdot T_P$$

Nei circuiti ad alta velocità questo è un limite importante: aggiungere Flip-Flop obbliga a rallentare il clock.

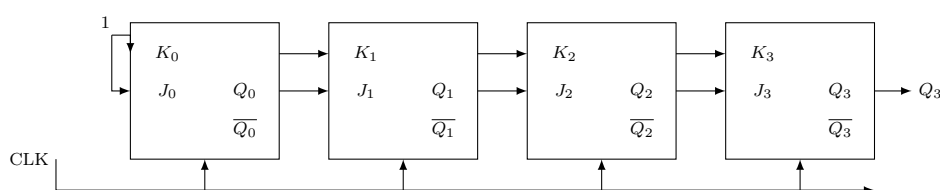


CONTATORE ASINCRONO

7.2.2 Contatore sincrono

Nel contatore sincrono, il clock **arriva contemporaneamente a tutti i Flip-Flop**, eliminando il ritardo a cascata e rendendo il contatore molto più veloce.

Come si implementa la logica di conteggio? Non tutti i FF devono commutare ad ogni tick: Q_1 deve commutare solo quando $Q_0 = 1$; Q_2 solo quando $Q_0 = Q_1 = 1$; e così via. Questa condizione viene realizzata con **porte AND in cascata**: l'ingresso J, K del FF i -esimo è connesso all'uscita di una porta AND che combina tutti i bit $Q_0, \dots, Q_{i-1}$. Il FF i riceve il permesso di commutare solo quando tutti i bit meno significativi valgono 1.



CONTATORE SINCRONO

7.2.3 Azzeramento dopo un certo conteggio (contatore modulo-M)

Spesso non si vuole un contatore che vada da 0 a $2^n - 1$, ma che si fermi ad un numero specifico M e poi riparta da zero.

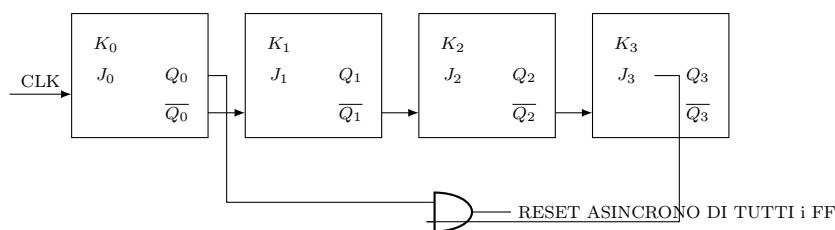
Implementazione: si aggiunge una porta AND ai cui ingressi arrivano le uscite Q_i che sono a 1 nella rappresentazione binaria di M . Non appena il contatore raggiunge M , l'uscita della porta AND va a 1 e genera un segnale di **RESET asincrono** che azzer immediatamente tutti i FF riportandoli a $Q = 0000$.

Attenzione al glitch sull'ultimo conteggio: il valore M compare realmente in uscita per un brevissimo istante (il tempo necessario affinché la porta AND lo riconosca e il reset arrivi) prima che tutto sia azzerato. L'ultimo stato ha quindi una durata anomala:

$$T_{\text{stato } M} = T_f(\text{JK-FF}) + T_P(\text{AND})$$

Esempio: contatore modulo-6 (conta 0, 1, 2, 3, 4, 5 e poi riparte da 0).

Quando $Q_3Q_2Q_1Q_0 = 0101$ (decimale 5), la porta AND rileva questo stato e genera il RESET, riportando tutto a 0000.



7.2.4 Ripple Carry Output (RCO)

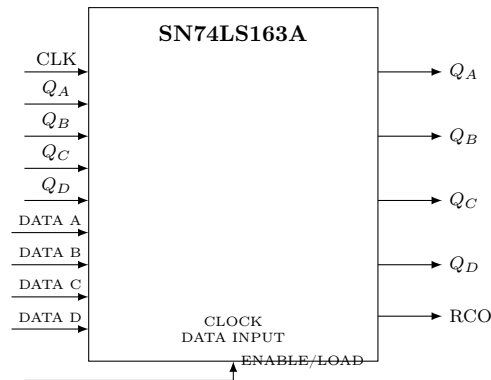
I contatori binari integrati (come il classico **SN74LS163A**) includono un'uscita speciale chiamata **RCO** (Ripple Carry Output).

A cosa serve? Quando si collegano più chip in cascata per formare un contatore a più bit (es. 8, 12, 16 bit), ogni chip deve segnalare al successivo quando ha terminato il proprio ciclo di conteggio. Il chip delle unità segnala al chip delle decine; il chip delle decine segnala a quello delle centinaia; e così via.

L'RCO è l'uscita di una porta NOR con 5 ingressi (i 4 $\overline{Q_i}$ e il segnale di *enable*): va a 1 per esattamente **un periodo di clock** quando il contatore ha raggiunto il valore massimo (tipicamente $Q = 1111 = 15$). Solo in quel preciso istante tutti i $\overline{Q_i}$ sono a 0, e la NOR dà 1:

$RCO = 1$ per un periodo T_{CLK} quando il conteggio è al massimo.

Il chip successivo nella catena usa questo impulso RCO come *enable* (o come clock) per incrementare il proprio conteggio: in questo modo si realizzano contatori a 8, 12, 16 bit collegando semplicemente chip da 4 bit in cascata tramite RCO.

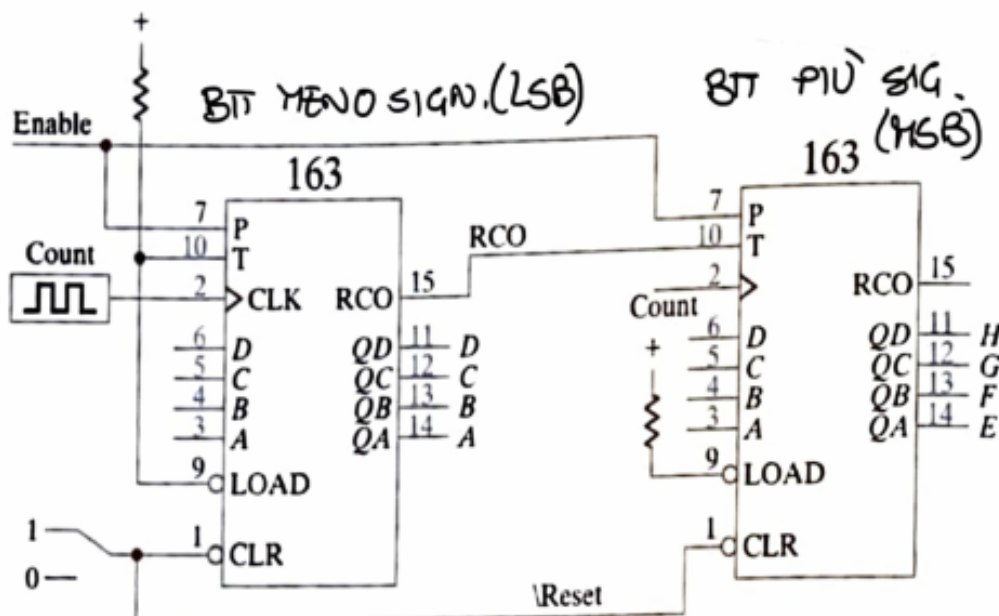


Capitolo 8. Conteggio, divisori di frequenza e PRFG

(20 aprile 2026)

8.1 Contatori a 8 bit: cascata di due SN74LS163

Due contatori sincroni a 4 bit si collegano in **cascata** tramite il segnale RCO: il bit più significativo (*MSB*, gestito dal secondo 163) viene incrementato quando il contatore meno significativo (*LSB*) raggiunge il valore 15 e porta RCO = 1. Il segnale RCO del primo modulo funge da *enable di conteggio* per il secondo.



Due contatori sincroni a 4 bit si collegano a cascata con il RCO che funge da *enable di conteggio*: il contatore MSB avanza di un'unità ogni volta che il LSB arriva a 15 (RCO = 1). In questo modo si ottiene un contatore a 8 bit capace di rappresentare i valori da 0 a 255 ($2^8 - 1$). Le uscite del secondo modulo (H, G, F, E) sono i bit più significativi dell'intero conteggio; il segnale Reset asincrono è comune ad entrambi i moduli.

8.2 Divisori di frequenza per un numero n

(20 aprile 2026)

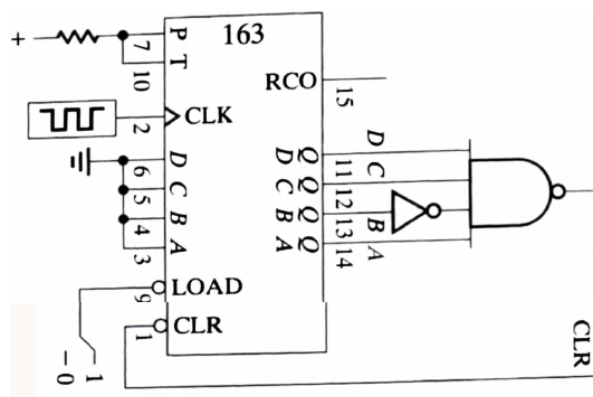
Divisore di frequenza = circuito sequenziale che prende in ingresso il segnale di clock e ne riduce la frequenza di un fattore n :

$$f_{\text{out}} = \frac{f_{\text{CLK}}}{n} \quad T_{\text{out}} = n T_{\text{CLK}}$$

Si realizzano con il contatore SN74LS163 sfruttando il segnale RCO o il LOAD. Esistono due varianti principali.

8.2.1 Divisore con CLEAR sincrono

Il circuito conta fino a DCBA = 1101 (13); dopodiché il CLEAR è 1 e ricomincia da capo.

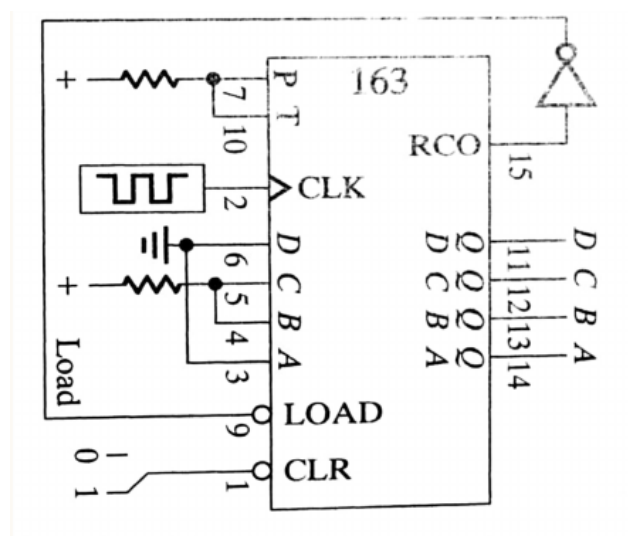


- A (LSB) ha $T_A = 2T_{CLK}$.
- D (MSB) ha $T_D = 16T_{CLK}$.
- Il numero 13 è l'ultimo contato, con $f = f_{CLK}/14$.
- Senza il clear sincrono l'ultimo stato (13) dura poco, non è esattamente un fronte.

Funzionamento: le uscite $Q_D Q_C Q_B Q_A$ vengono decodificate da una porta AND (con buffer invertente sul segnale C per selezionare il pattern 1101); quando l'AND vale 1, il segnale CLR si attiva e azzer il contatore al colpo di clock successivo. L'uscita CLR torna quindi a 0 e il ciclo ricomincia.

8.2.2 Divisore con SET sincrono (LOAD)

Il circuito ha il RCO collegato al LOAD (set) in modo che, dopo aver finito di contare, venga caricato il valore impostato su DCBA = 0110 (6).



- RCO ha una frequenza di 4/ciclo T_{CLK} (1/10).
- In generale: $T_s = (2^n - n_{load}) T_{CLK}$.

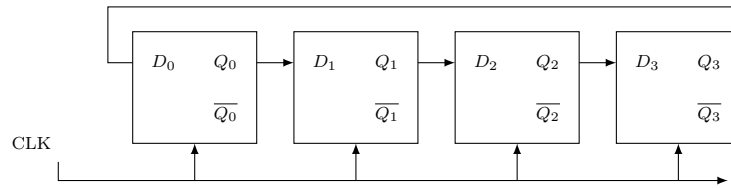
Funzionamento: quando il contatore raggiunge 15 e $RCO = 1$, al colpo di clock successivo il LOAD forza il caricamento del valore preset (DCBA = 0110 in questo esempio); il contatore riparte da 6 anziché da 0, ottenendo un ciclo di lunghezza $16 - 6 = 10$, cioè $f_{out} = f_{CLK}/10$. Il LED collegato all'RCO si accende una volta per ciclo.

8.3 Contatore ad anello

Si pone in ingresso $Q = 0001$ e si applica il clock per vedere come evolve:

$$0001 \rightarrow 0010 \rightarrow 0100 \rightarrow 1000$$

I FF copiano il valore in entrata: contatore con solo **4 valori**.

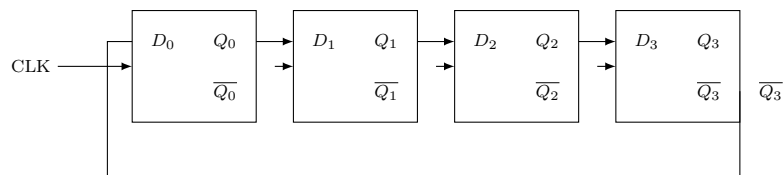


8.4 Contatore ad anello *twisted* (Johnson counter)

Si pone in ingresso $Q = 0000$ e si applica il clock; si nota che $\overline{Q_3}$ è collegato a D_0 (feedback invertito):

$0000 \rightarrow 0001 \rightarrow 0011 \rightarrow 0111 \rightarrow 1111 \rightarrow 1110 \rightarrow 1100 \rightarrow 1000 \rightarrow 0000$

I FF copiano il valore in entrata: contatore con **8 valori** (con 4 FF).



Rispetto al contatore ad anello normale, il Johnson counter ha **$2N$ stati** per N flip-flop utilizzati per realizzarlo.

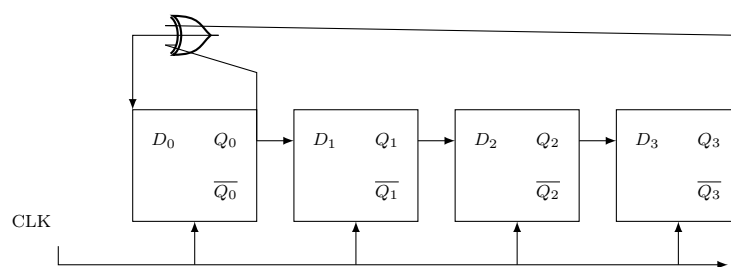
8.5 Generatori di frequenza pseudo casuali (PRFG)

PRFG (*Pseudo-Random Frequency Generator*) = applicazione dei registri a scorrimento che ha trovato grande applicazione nella generazione di sequenze di bit pseudo casuali. Il registro ha grandezza fissa; questo comporta lo “pseudo”.

Un registro largo K FF genera una sequenza ripetuta al massimo dopo $2^K - 1$ eventi.

Nei generatori pseudo casuali **lineari** si usa lo XOR (o XNOR) per costruire il circuito: le porte equivalenti alla somma binaria.

LPRG = *Linear Pseudo Random Generator*



La tabella degli stati del LPRG a 3 FF (FF0, FF1, FF2), con feedback XOR tra FF2 e FF1, mostra la sequenza pseudo casuale di lunghezza $2^3 - 1 = 7$:

Clock	FF0	FF1	FF2
0	1	1	1
1	0	1	1
2	1	0	1
3	1	1	0
4	0	1	1
5	1	0	1
6	0	0	1
7	1	0	0
8	1	1	0
9	0	1	1
10	0	0	1
11	1	0	0
12	1	1	0
13	1	1	1

Capitolo 9. Macchine a stati finiti (FSM)

(24 aprile 2026)

FSM (*Finite State Machine*) = architettura in cui gli stati presenti determinano quelli futuri.

Diagramma di stato = mappa per rappresentare il comportamento di un sistema sequenziale (possibili stati e legami).

Gli stati sono legati fra loro da precise transizioni. Gli stati sono definiti da condizioni **INTERNE** (presenti nel sistema) e/o da condizioni **ESTERNE** (segnali in ingresso dall'esterno).

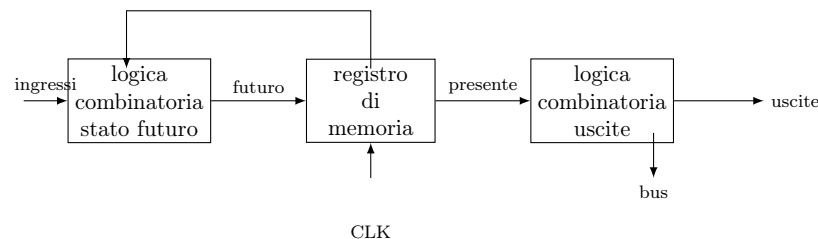
Ogni circuito digitale è una FSM.

9.1 Tipi di architettura FSM: Moore e Mealy

Esistono due tipi principali di architettura per le FSM:

- Architettura di Moore
- Architettura di Mealy

9.1.1 Architettura di Moore

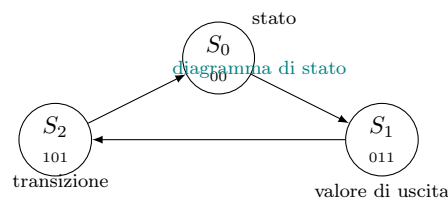


- Uno stato è univocamente conseguenza dello stato precedente.
- Gli ingressi esterni non sempre sono presenti.

9.2 Dal diagramma di stato al circuito

(24 aprile 2026)

Il diagramma di stato è composto da **stati** (cerchi etichettati con il valore di uscita) collegati da **transizioni** (freccie etichettate con il valore dell'ingresso/condizione):



La procedura per passare dal diagramma di stato al circuito è la seguente:

1. **Dato il numero di stati** (3) definisco il numero di bit che mi serve a rappresentarli: $\lceil \log_2 3 \rceil = 2 \Rightarrow 2$ bit.
2. **Associo a ogni stato una rappresentazione in bit**: $S_0 = [00]$, $S_1 = [01]$, $S_2 = [10]$, $S_3 = [X]$ (*don't care*).

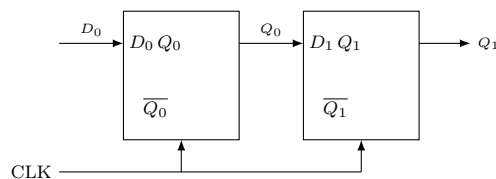
3. Per ogni bit uso un FF: $S_i = Q_i Q_0^i$.
4. Scrivo la tabella di verità: associo a ogni stato presente uno stato futuro.

Q_1	Q_0	Q_1^+	Q_0^+
0	0	0	1
0	1	1	0
1	0	0	0
1	1	×	×

5. Scelgo il tipo di FF (D, JK, T): in questo caso uso DFF, poiché $Q^+ = D$.
6. Ricavo le funzioni di $D_i = f(Q_i)$:

$$D_1 = Q_0 \quad D_0 = \overline{Q_1} \cdot \overline{Q_0}$$

7. Disegno il circuito:



- **DFF**: $Q^+ = D$ più usato, semplice.
- **TFF**: $Q^+ = T \oplus Q$ per contatori e sequenze regolari.
- **JKFF**: flessibile, può fare tutto (set, reset, toggle).

9.2.1 Ricavare le funzioni dei segnali di uscita: $O_i = f(Q_i)$

Q_1	Q_0	D_0	D_1	O_2	O_1	O_0
0	0	1	0	0	1	0
0	1	0	1	0	1	1
1	0	0	0	1	0	1
1	1	×	×	×	×	×

$$O_0 = Q_0 + Q_1 \quad O_1 = \overline{Q_1} \quad O_2 = Q_1$$

9.2.2 Stato bloccante e segnale di Enable

Al punto 9 si verifica quale sia lo stato futuro del caso $Q_1 = Q_0 = 1$: se è uno **stato bloccante**, è un problema che dobbiamo risolvere.

Nel nostro caso, da $Q = 11$ si andrà a finire in $D = 10$, quindi ricomincerà la sequenza (non blocca il circuito).

Se volessimo scorrere il ciclo al contrario, possiamo inserire un segnale di **Enable**:

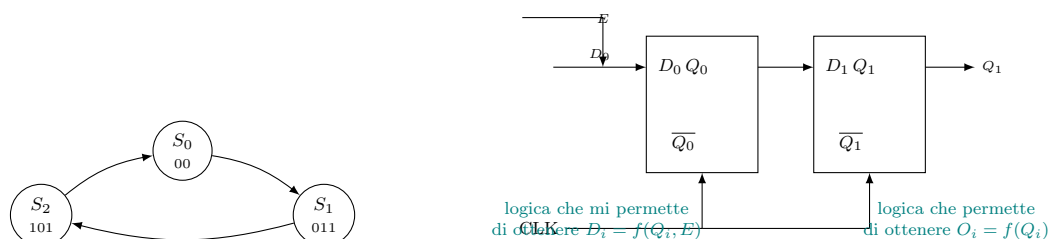
- $E = 1$: si ha $010 \rightarrow 011 \rightarrow 101$ (dritto).
- $E = 0$: si ha $101 \rightarrow 011 \rightarrow 010$ (rovescio).

Cambiamo le entrate D_i ma le uscite restano le stesse:

$$D_i = f(Q_i, E) \quad O_i = f(Q_i)$$

Possiamo scrivere le funzioni:

$$D_1 = \overline{E} \cdot \overline{Q_1} \cdot \overline{Q_0} + E \cdot Q_0 \quad D_0 = E \cdot \overline{Q_1} \cdot \overline{Q_0} + \overline{E} \cdot Q_1$$



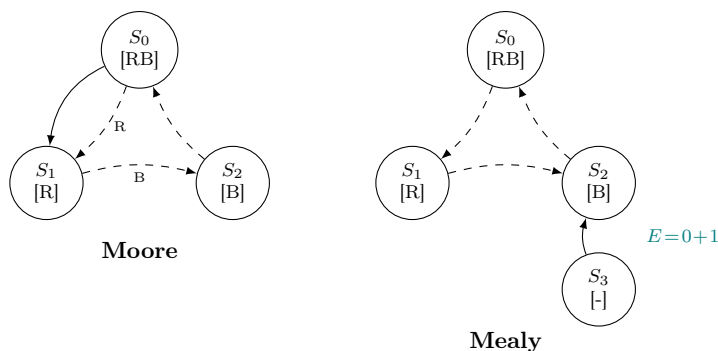
9.2.3 Esempio: insegna luminosa (continuazione)

La sequenza dell'insegna luminosa, con $E = 1$, è:

$$RABA \rightarrow RABS \rightarrow RSBA \rightarrow RABA \rightarrow \dots \quad (E = 1)$$

$$BA \rightarrow BS \rightarrow BA \rightarrow \dots \quad (E = 0, \text{RS costante})$$

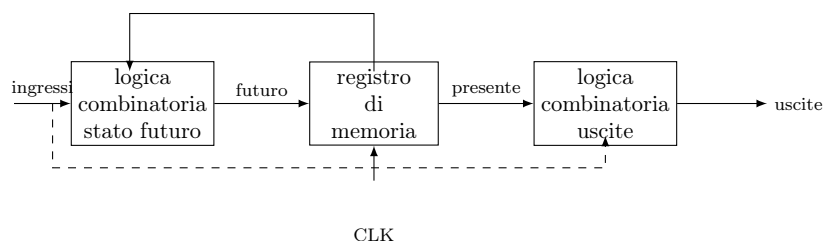
Per aggiungere lo stato “RS costante” disabilitato, dobbiamo aggiungere un 4° stato, perché $O_i \leftrightarrow f(Q_i) \neq E$.



Possiamo quindi implementare l'architettura di Moore.

9.3 Architettura di Mealy

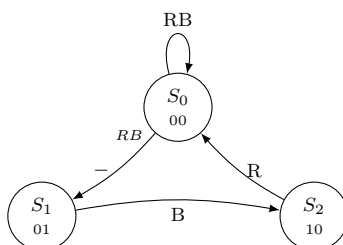
(27 aprile 2026)



La differenza principale fra Moore e Mealy è la dipendenza delle uscite dai parametri del circuito:

- in **Moore**: $O_i = f(Q_i)$, sincrona al clock.
- in **Mealy**: $O_i = f(Q_i, I_i)$, asincrona.

Il valore delle uscite si associa alle **transizioni**. Riprendendo l'esempio dell'insegna luminosa, il diagramma di Mealy è:



- $E = 1$: l'uscita è legata al passaggio fra 2 stati, non allo stato di per sé.
- $E = 0$: il fronte di clock è asincrono, perché dipende anche da E , che è asincrono: se il clock si ferma ma E cambia, allora le uscite cambiano.

Per implementare il circuito dei registri si usano JKFF in modalità “toggle” (1) oppure “hold” (0): $J = K$ in una sola modalità.

9.3.1 Tabella di verità dell'insegna luminosa (Mealy)

Q_1	Q_0	stato futuro		uscite	
		Q_1^+	Q_0^+	R	B
$E = 1$					
0	0	0	1	1	1
0	1	1	0	1	0
1	0	0	0	0	1
1	1	×	×	×	×
$E = 0$					
0	0	0	0	1	0
0	1	0	1	0	1
1	0	0	1	0	1
1	1	×	×	×	×

9.3.2 Tabella JK per l'implementazione

Usando JKFF con $J = K$ (modalità toggle quando $Q_i \neq Q_i^+$, hold quando $Q_i = Q_i^+$):

Q_1	Q_0	Q_1^+	Q_0^+	JK_1	JK_0
$E = 1$					
0	0	0	1	0	1
0	1	1	0	1	1
1	0	0	0	1	0
1	1	×	×	×	×
$E = 0$					
0	0	0	0	0	0
0	1	0	1	0	0
1	0	0	1	1	1
1	1	×	×	×	×

- Quando $Q_i = Q_i^+$: fa **hold** ($JK = 0$).
- Quando $\overline{Q_i} = Q_i^+$: fa **toggle** ($JK = 1$).

$$Q_1^+ = E Q_0 \quad Q_0^+ = \overline{Q_0} (\overline{E} + \overline{Q_1})$$

9.3.3 Mappe di Karnaugh per JK_0 e JK_1

Mappa JK_0					Mappa JK_1				
$E \backslash Q_1 Q_0$	00	01	11	10	$E \backslash Q_1 Q_0$	00	01	11	10
0	0	0	×	0	0	0	0	×	1
1	1	1	×	0	1	0	1	×	1

$JK_0 = \overline{Q_1} + \overline{E}$ $JK_1 = Q_1 + E Q_0$

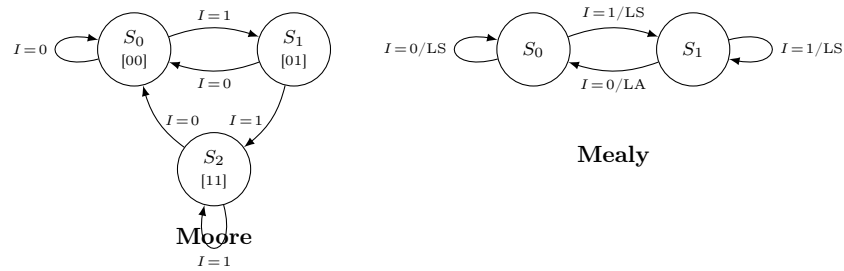
9.4 Esempio: riconoscitore di sequenze

In ingresso c'è una serie di bit: vogliamo accendere un LED quando trovo $Q_{i+1}Q_i = 11$:

Serie di bit: $I = 10110101110001\dots$ **LED acceso**

- S_0 = non ho ancora trovato il 1° 1: LED spento (S).
- S_1 = ho trovato il 1° 1: LED spento (S).
- S_2 = ho trovato il 2° 1: LED acceso (A).

- $I = 1, I = 0$.



9.5 Esempio: macchina distributrice del caffè

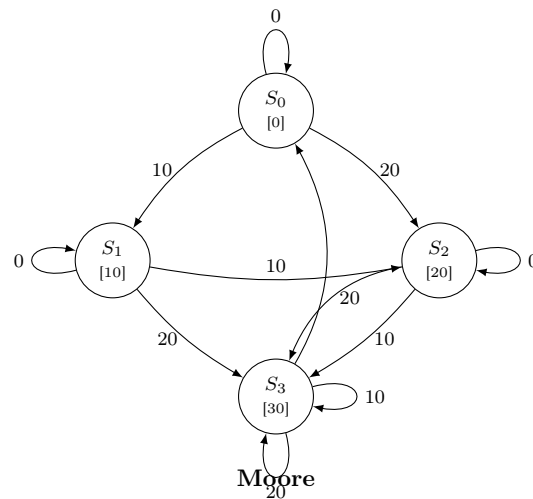
- Costo caffè: 0,30 €.
- L'erogazione del caffè avviene se metto $\geq 0,30$ €.
- La macchina **non dà resto**.
- La macchina riconosce il peso e accetta solo monete da 0,10 € e 0,20 €.

Stati della macchina (2 FF, 4 stati):

Stato	Credito
S_0	0 cent
S_1	10 cent
S_2	20 cent
S_3	≥ 30 cent

9.5.1 Diagramma di stato della macchina del caffè (Moore)

Per Mealy ogni stato non deve avere uscite mai definite, altrimenti ci sono dei problemi di confusione.



Capitolo 10. Elettronica combinatoria complessa

(11 maggio 2026)

Circuiti che permettono di implementare funzioni in modo compatto.

10.1 ROM – Read Only Memory

ROM (*Read Only Memory*): può essere interpretata in 2 modi:

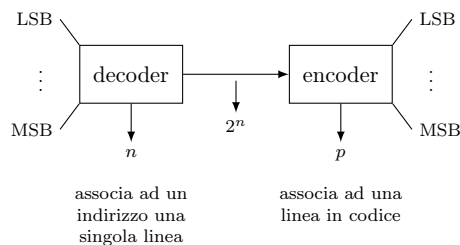
1. **Memoria** da 2^n parole da p bit.
2. **Implementazione** di p funzioni delle n variabili in ingresso.

Per n linee di ingresso ho 2^n possibili combinazioni; p sono le linee di uscita.

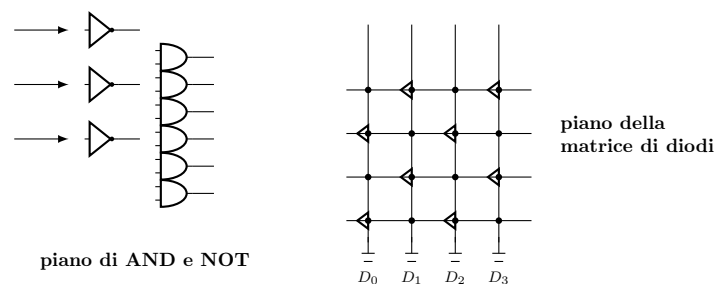


10.1.1 Come è fatta?

La ROM è costituita da un **piano di AND** e un **piano di OR**:



Per realizzare il circuito si usa la **matrice di diodi**.



La matrice di diodi usa **diodi** (non resistenze, così non c'è influenza fra linee diverse). Possiamo vedere $B_i = f(T_j)$; ad esempio:

$$B_0 = T_1 + T_3 + T_5 + T_7 + T_9 \quad (\text{righe collegate alla colonna } B_0)$$

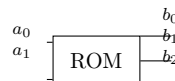
Il circuito che implementa la ROM è: **decoder + encoder**.

10.1.2 Esempio: la tastiera

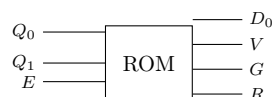
Ad ogni tasto si associa un codice BCD. La tastiera ha tasti $T_0 \dots T_9$ e uscite $B_3 B_2 B_1 B_0$ (codice BCD a 4 bit):

	T_9	T_8	T_7	T_6	T_5	T_4	T_3	T_2	T_1	T_0	B_3	B_2	B_1	B_0
0	...	—	—						0	1	0	0	0	0
...									...	...	...	...	...	...
1	0	—							0	1	0	1	0	0

Gli incroci non sono collegati: i rami vengono uniti tramite la **matrice di diodi**.

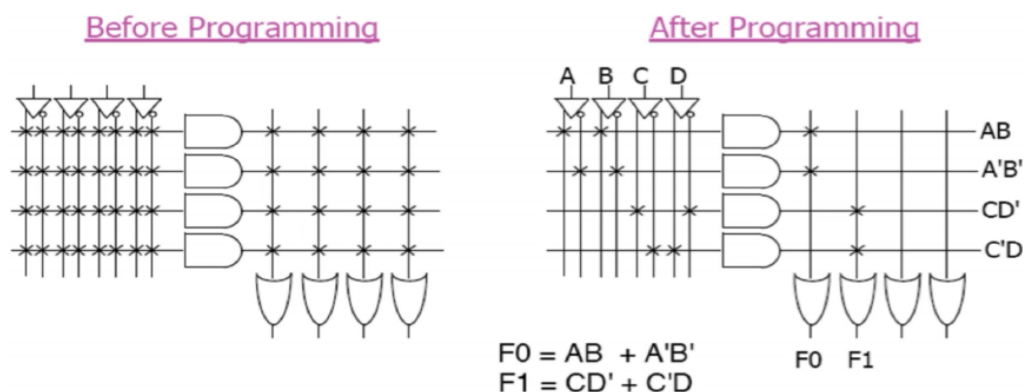


10.1.3 Esempio: la ROM del semaforo



10.1.4 Struttura della PLA: prima e dopo la programmazione

La PLA è composta da due matrici programmabili: un **piano AND** (che genera gli implicanti selezionati) e un **piano OR** (che combina gli implicanti per ciascuna uscita). Prima della programmazione *tutti* gli incroci sono connessi; la programmazione **brucia i diodi inutili**, lasciando solo le connessioni che implementano le funzioni desiderate.



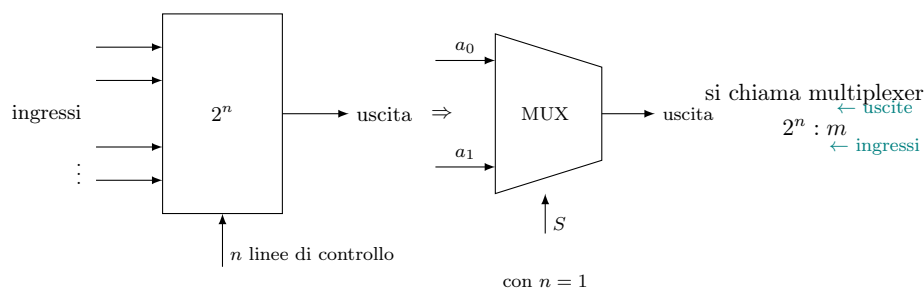
Esempio: con 4 ingressi A, B, C, D e 4 implicanti programmati ($AB, A'B', CD', C'D$) si implementano le due funzioni:

$$F_0 = AB + A'B' \quad F_1 = CD' + C'D = C \oplus D$$

La **PLA** risparmia area rispetto alla ROM perché genera *solo* gli implicanti necessari e non tutti i 2^n mintermini. Il numero di righe del piano AND è uguale al numero di implicanti distinti scelti (non a 2^n).

10.2 Multiplexer e demultiplexer

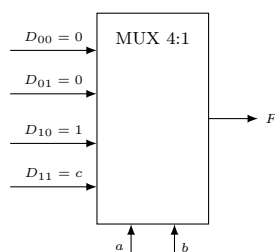
MUX (multiplexer) = circuito digitale che funziona similmente a un selettore, ma con 2^n ingressi, n linee di controllo e 1 uscita.



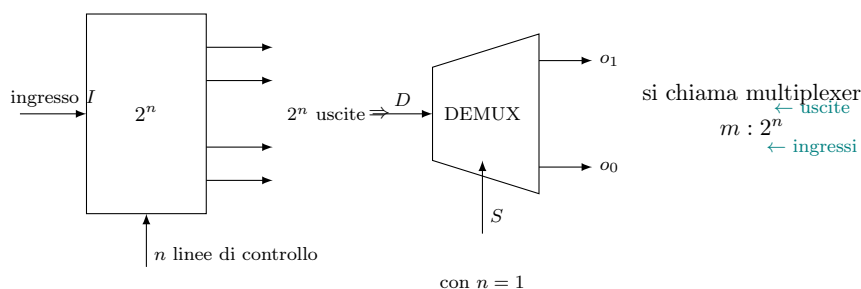
Ogni funzione a n variabili può essere implementata con un multiplexer $2^{n-1} : 1$.

Consideriamo $F(a, b, c) = a\bar{b} + abc$ e costruiamo la **tabella di verità funzionale** usando a e b come indirizzi del MUX 4:1 ($2^2 = 4$ ingressi) e c come eventuale ingresso dati:

a	b	D_i (MUX input)
0	0	0
0	1	0
1	0	1
1	1	c

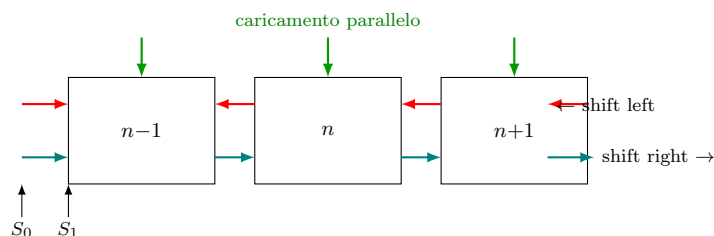


DEMUX (demultiplexer) = circuito digitale opposto al MUX, con 1 ingresso, n linee di controllo e 2^n uscite.



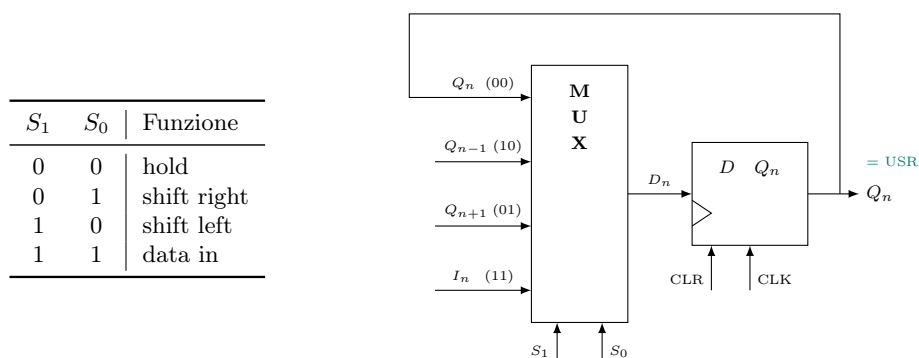
10.3 Universal Shift Register (USR)

USR (Universal Shift Register) = dispositivo che permette di mettere dati in parallelo o in serie e di spostarli a destra e sinistra.



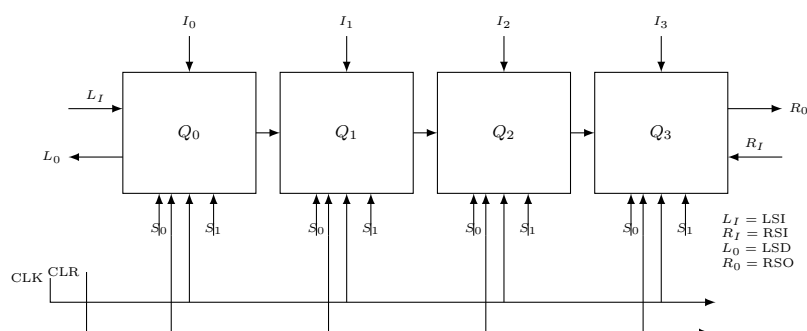
Vediamo come implementare ogni singola unità dell'USR.

Possiamo guidare le quattro funzioni con un **MUX 4:1** (2 indirizzi): HOLD, SHIFT LEFT, SHIFT RIGHT, DATA IN.



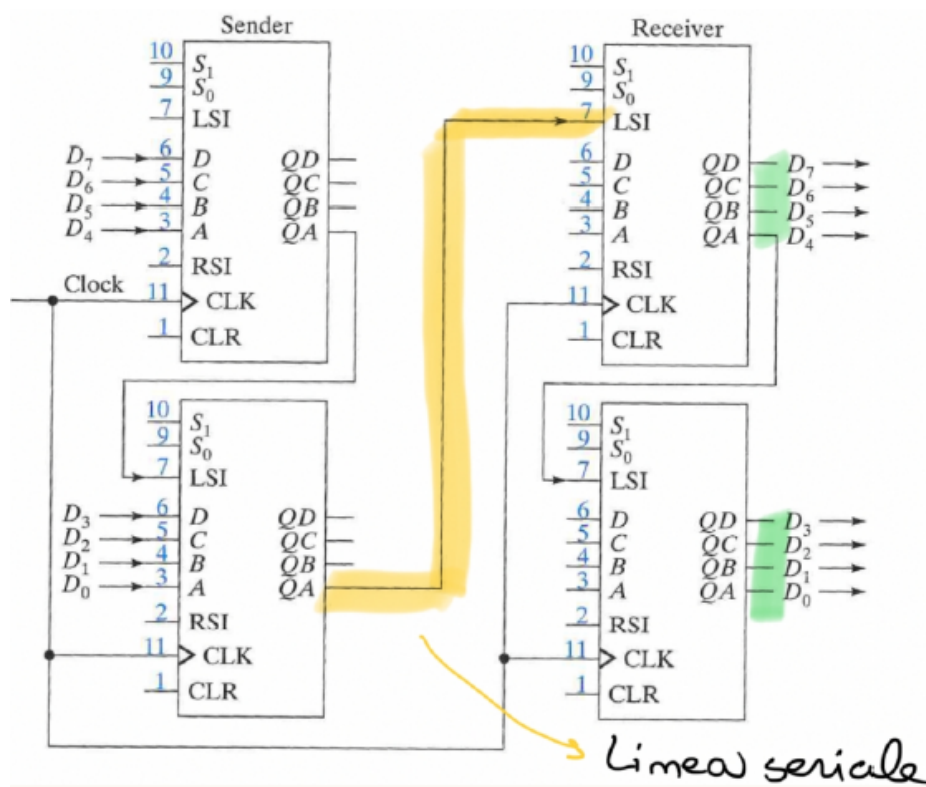
Se $CLR = 1$ si effettua un RESET a 0 sincrono.

10.3.1 Implementazione a 4 bit per trasmissione PISO



Il circuito permette 4 modalità operative selezionate da S_1S_0 : caricamento parallelo, shift left (seriale verso sinistra), shift right (seriale verso destra) e hold.

10.3.2 Applicazione: trasmissione seriale Sender-Receiver



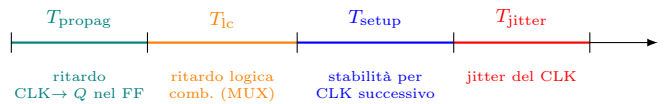
Il flusso dell'operazione è:

Caricamento nel Sender di $D_i \xrightarrow{\text{shift}}$ Trasmissione sulla linea seriale $\xrightarrow{\text{shift}}$ Lettura sulle uscite del Receiver

10.4 Vincoli temporali dell'USR e Jitter

La **massima frequenza del clock** per un USR corrisponde al **minimo periodo del clock**:

$$T_{\text{CLK,min}} = T_{\text{setup}} + T_{\text{propag}} + T_{\text{lc}} + T_{\text{jitter}}$$



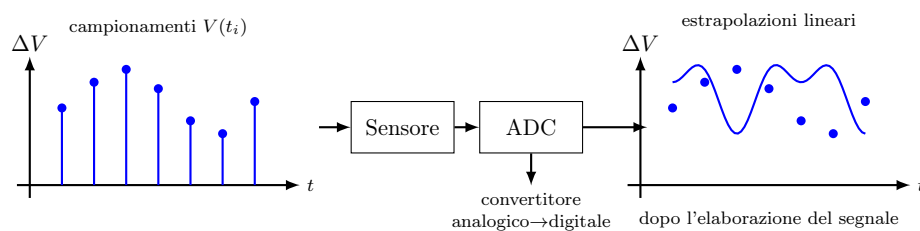
Jitter del CLK = Δ_{max} fra gli istanti di arrivo dei fronti di clock che dovrebbero giungere *contemporaneamente* a tutti i componenti del circuito, ma che in pratica non lo fanno a causa di variazioni fisiche (disadattamenti di linea, rumore, disomogeneità dei percorsi).

Capitolo 11. Conversione Digitale–Analogica e Analogica–Digitale

(15 maggio 2026)

11.1 Conversione digitale ↔ analogica

Il processo di acquisizione e ricostruzione del segnale si articola in due fasi:



Il quesito è: cosa succede fra il campionamento e la ricostruzione? La risposta è **sample and hold** → **convertitore analogico→digitale** → **convertitore digitale→analogico**, in sintesi:

$$\text{Sensore} \xrightarrow{\text{ADC}} \text{elaborazione digitale} \xrightarrow{\text{DAC}} \text{uscita analogica}$$

Definizioni di base:

- $n_{(.)}$ = numero che dipende dai bit disponibili;
- ΔV = **granularità**, ovvero il passo minimo distinguibile;
- V_{ref} = valore massimo attingibile.

Funzione di trasferimento di un ADC: $n_{\text{out}} = V_{\text{in}} / \Delta V$

Funzione di trasferimento di un DAC: $V_{\text{out}} = n_{\text{in}} \cdot \Delta V$

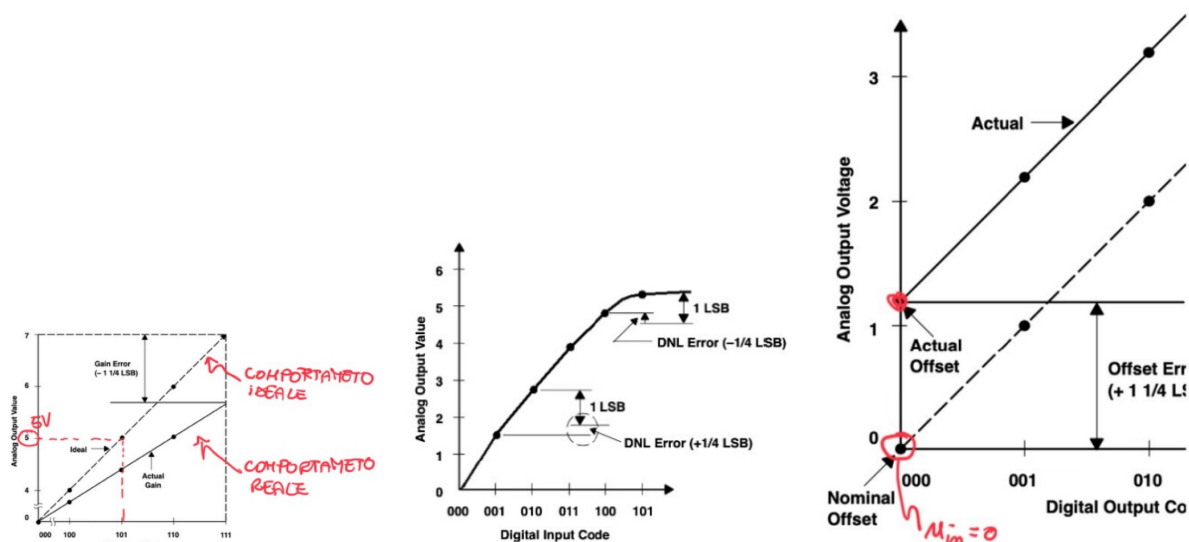
11.2 Caratteristiche del DAC

Precisione = determinata dal numero di bit e dall'intervallo di tensioni:

$$V_{\text{out}} = n_{\text{in}} \cdot \Delta V$$

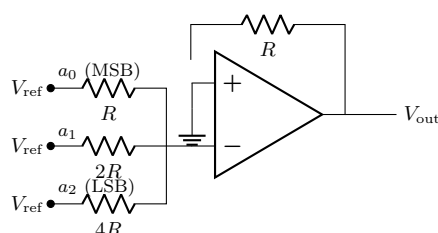
Ad esempio, con $n\text{-bit} = 3$ e $\Delta V = 1\text{ V}$: $n_{\text{in}} \in [0, 2^3 - 1] \Rightarrow V_{\text{out}} \in [0, 2^3 - 1]\text{ V}$.

La velocità di conversione è molto importante e dipende da diversi fattori: tipo di assestamento, architettura e numero di bit. Le tre principali sorgenti di errore nel DAC sono:



11.3 DAC a sommatore

L'architettura più semplice di DAC è il **DAC a sommatore pesato**: si usa un amplificatore operazionale invertente con resistenze pesate $R_i = R/2^i$ per ciascun bit. Le resistenze di ingresso sono collegate a V_{ref} (bit $a_i = 1$) o a massa (bit $a_i = 0$) tramite uno switch digitale per ciascun bit.



Nota: ogni ramo a_i è collegato a V_{ref} tramite uno switch digitale, che si chiude o si apre (collegando in alternativa a massa) a seconda del valore del bit; il simbolo del cursore $\updownarrow$ negli appunti originali indica proprio questo switch a monte dell'ingresso invertente.

La formula di uscita si ricava dalla sovrapposizione degli effetti:

$$V_{out} = -V_{ref} \left(\frac{R}{R_0} + \frac{R}{R_1} + \frac{R}{R_2} \right)$$

Se si pone $R_i = R/2^i$, allora $R/R_i = 2^i$ e quindi:

$$V_{out} = -V_{ref} \sum_i 2^i$$

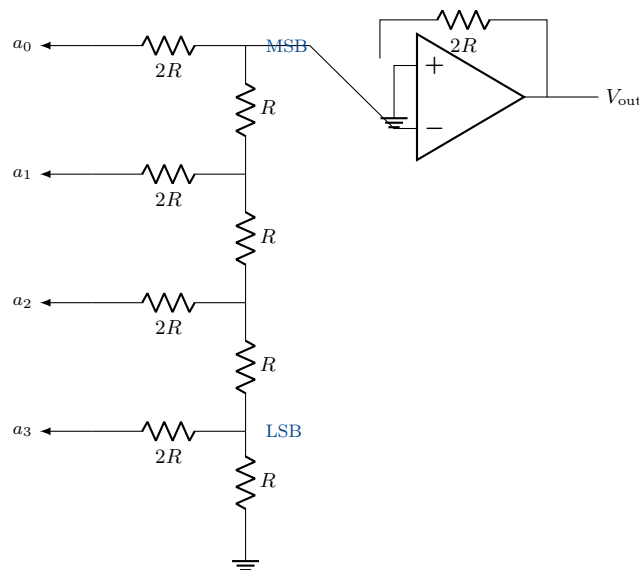
Considerando gli switch digitali, ogni bit a_i vale 0 (collegamento a terra) oppure 1 (collegamento a V_{ref}):

$$V_{out} = -V_{ref} \sum_i a_i 2^i$$

Limite pratico: supponendo 10 bit, $R_i = 100 \text{ k}\Omega/2^i$. Se la tolleranza è $\sigma_0 = 1\%$ e $R_0 = 1 \text{ k}\Omega$, i bit meno significativi vengono persi perché il loro valore di resistenza scende sotto la tolleranza stessa ($R_i < \sigma_0$). Questo DAC è quindi utilizzabile solo con pochi bit; altrimenti non è sufficientemente preciso.

11.4 DAC R-2R Ladder

Per avere un DAC più preciso si usa la tecnica **R-2R Ladder**, che genera pesi binari per i bit usando solo due valori di resistenza (R e $2R$):



Ogni bit a_i contribuisce all'uscita con un peso binario:

$$a_i \rightarrow V_{\text{out},i} = -\frac{V_{\text{ref}}}{2^{i+1}} \Rightarrow \text{i pesi sono } \frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \frac{1}{16}, \dots$$

$$V_{\text{out}} = -V_{\text{ref}} \sum_i \frac{a_i}{2^{i+1}}$$

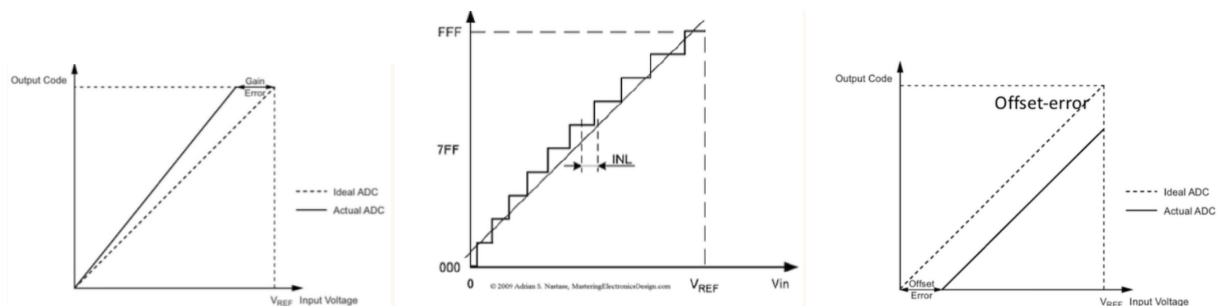
Esempio: con 4 bit, ingresso 1010_2 e $V_{\text{ref}} = 8 \text{ V}$:

$$V_{\text{out}} = -8 \left(\frac{1}{2} + \frac{1}{8} \right) = -8 \cdot \frac{5}{8} = -5 \text{ V}$$

11.5 Caratteristiche dell'ADC

- **Accuratezza** = insieme di tutte le sorgenti di errore, errore di quantizzazione compreso.
- **Errore di quantizzazione** = dipende dal tipo di ADC ed è dovuto alla discretizzazione del segnale durante la conversione; di solito vale 1 LSB oppure 0,5 LSB.
- **Tempo di conversione e funzionamento** sono altre caratteristiche importanti dell'ADC.

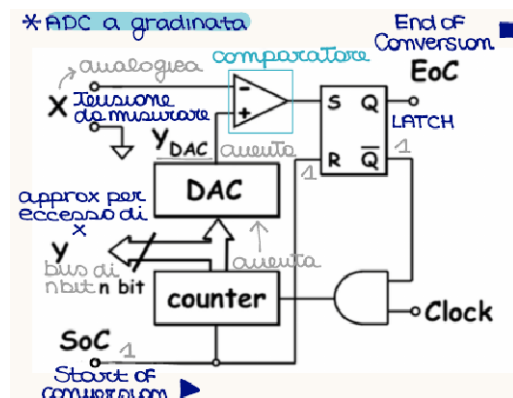
Analogamente al DAC, anche l'ADC presenta tre principali tipi di errore rispetto al comportamento ideale:



Capitolo 12. Convertitori Analogico-Digitali (ADC)

(15 maggio 2026 – cont.)

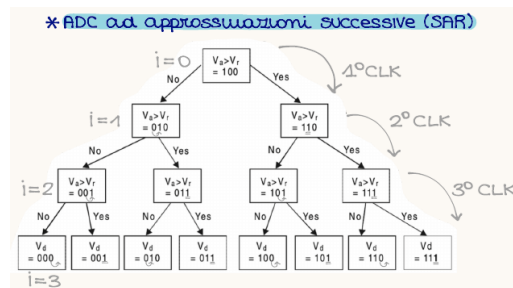
12.1 ADC a gradinata



- **Funzionamento del comparatore:** finché $X > Y_{DAC}$ il conteggio procede; appena $Y_{DAC} \geq X$ il comparatore cambia stato → il latch RS blocca il CLK e il counter.
- **Funzionamento del contatore:** genera $N_{in2} \rightarrow N_{in0}$ e aumenta la tensione.
- La granularità dipende da V_{REF} del DAC (cioè il ΔV con cui aumenta).
- Asintoticamente questo ADC deve fare 2^n confronti, dove n sono i livelli del DAC:

$$T_{conv} = 2^n T_{CLK}$$

12.2 ADC ad approssimazioni successive (SAR)

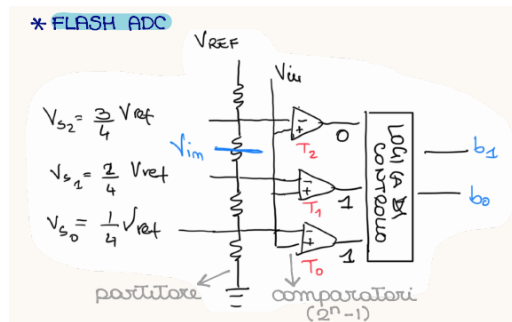


- Il valore di tensione da misurare si confronta con i successivi.
- Ogni conversione avviene in un ciclo di CLK. Se $n_{bit} = 3$ ci sono 3 cicli.
- **Yes:** metto un 1 nella posizione $i + 1$ se passo da i a $i + 1$.
- **No:** sposto l'1 dalla posizione i alla posizione $i + 1$ passando da i a $i + 1$.
- È una ricerca binaria: i confronti sono n , quindi:

$$T_{conv} = n T_{CLK}$$

- Funzionamento specifico alla pagina successiva.

12.3 Flash ADC



- La tensione viene confrontata contemporaneamente con n valori; poi un circuito combinatorio converte le uscite dei confronti in binario (1 = valori coperti da V_{in} ; 0 = valori più alti di V_{in}).
- Un confronto: istantaneo (fatto in parallelo) $\Rightarrow$ è il più veloce.
- La precisione è però molto minore di quella degli altri (servono tante resistenze, con i problemi già discussi).
- Risolvo coi mintermini, sapendo che se $j < i < k$ e $T_i = 1$ allora $T_j = 1$ e $T_k = 0$:

$$b_0 = T_2 + T_0 \cdot T_1$$

$$b_1 = T_2 + T_1 \cdot \overline{T_2}$$

12.4 Osservazioni generali

- L'errore di quantizzazione è pari a $0,5 V_{REF}/2^n$.

12.5 ADC dell'AD2

- Ci sono 14 bit $\Rightarrow 2^{14}$ possibili valori.
- Se scelgo la scala 0.5 V/div l'intervallo in cui misuro è $(0, 0,5 \cdot 10) \text{ V} = (0, 5) \text{ V}$, e la quantizzazione è:

$$\frac{5 \text{ V}}{2^{14}} \approx 0.3 \text{ mV}$$

12.5.1 ADC ad approssimazioni successive (SAR): analisi del circuito a 3 bit

Il convertitore analogico-digitale ad *approssimazioni successive* (in inglese **S**uccessive **A**pproximation **R**egister, SAR) determina il codice digitale corrispondente ad una tensione analogica V_a eseguendo un confronto "dicotomico" bit a bit, a partire dal bit più significativo (MSB) fino al meno significativo (LSB). Il circuito visto a lezione realizza un SAR a 3 bit ed è composto dai seguenti blocchi funzionali:

- **Ring Counter** (registro a scorrimento circolare) costituito da 5 flip-flop $S_0, \dots, S_4$: genera, un colpo di clock alla volta, un impulso che si sposta da uno stadio al successivo. Questo impulso scandisce il tempo di test di ciascun bit (MSB $\rightarrow$ LSB) e determina quando la conversione è terminata.
- **Registro dei bit di prova** composto da 3 flip-flop Q_2, Q_1, Q_0 (MSB $\rightarrow$ LSB): contiene, istante per istante, il codice binario di tentativo.
- **DAC** (convertitore digitale-analogico): converte il codice presente in $Q_2Q_1Q_0$ nella tensione di confronto V_{DAC} .
- **Comparatore analogico**: confronta V_a (tensione incognita da convertire) con V_{DAC} .
- **Logica combinatoria (porte AND/OR)**: in base all'uscita del comparatore e allo stadio attivo del Ring Counter, decide se mantenere a 1 oppure resettare a 0 il bit appena testato.

È importante notare che il Ring Counter e il registro dei bit di prova lavorano *in parallelo*: mentre il Ring Counter "punta" al bit che si sta testando in quel colpo di clock, il registro $Q_2Q_1Q_0$ contiene il codice di prova corrente, che viene aggiornato bit dopo bit.

Fase 1 – Inizializzazione (prima del primo fronte di clock)

Prima dell'arrivo del primo fronte di clock il circuito viene inizializzato nel seguente modo:

1. Si invia il segnale di **CLEAR** (attivo basso) contemporaneamente a tutti i flip-flop del registro dati Q_2, Q_1, Q_0 : tutte le uscite vengono forzate a 0.

- Contemporaneamente si setta $S_0 = 1$ (primo stadio del Ring Counter) tramite ingresso asincrono, mentre tutti gli altri stadi $S_1, \dots, S_4$ vengono inizializzati a 0.
- Poiché il registro dati è tutto a 0, l'ingresso del DAC è 000, quindi l'uscita del DAC è

$$V_{DAC} = 0 \text{ V.}$$

Lo stato del sistema subito prima del primo fronte di clock è dunque:

$$(S_4 S_3 S_2 S_1 S_0) = (00001), \quad (Q_2 Q_1 Q_0) = (000), \quad V_{DAC} = 0 \text{ V.}$$

Fase 2 – Subito dopo il primo fronte di clock

Al primo fronte di clock accadono due cose contemporaneamente:

- Il Ring Counter avanza di uno stadio: $S_0 \rightarrow 0$, $S_1 \rightarrow 1$. Questo impulso su S_1 abilita la scrittura del bit più significativo Q_2 .
- La logica combinatoria pone *a priori* $Q_2 = 1$ (si “prova” il bit più significativo settandolo a 1): il nuovo codice di prova è $Q_2 Q_1 Q_0 = 100$.

Con il nuovo codice in ingresso, il DAC produce la tensione corrispondente a $n = 4$ (in decimale):

$$V_{DAC} = V(n=4) \quad (\text{tensione corrispondente al codice } 100).$$

A questo punto il comparatore confronta V_a con V_{DAC} : il risultato di questo confronto sarà disponibile e utilizzato al fronte di clock *successivo* per decidere se mantenere $Q_2 = 1$ oppure resettarlo a 0.

Esempio numerico

Supponiamo che il DAC abbia una risoluzione $\Delta V = 1 \text{ V}$ sull'intervallo $[0, 7] \text{ V}$ (3 bit, 8 livelli) e che la tensione incognita sia

$$V_a = 5.5 \text{ V.}$$

Dopo il primo fronte di clock il registro contiene $Q_2 Q_1 Q_0 = 100$, quindi

$$V_{DAC} = 4 \text{ V.}$$

Il comparatore verifica se $V_a > V_{DAC}$ (equivalentemente, se V_a supera la metà del fondo scala, $V_{max}/2$):

$$V_a = 5.5 \text{ V} > V_{DAC} = 4 \text{ V} \implies \text{SI.}$$

Poiché il confronto ha dato esito positivo, il bit Q_2 appena testato viene **mantenuto a 1** (l'approssimazione “per difetto” era corretta: la tensione reale è maggiore di quella generata dal DAC con quel bit a 1, quindi quel bit resta valido nel codice finale).

Fase 4 – Subito dopo il secondo fronte di clock

Al secondo fronte di clock:

- Il Ring Counter avanza ancora: $S_1 \rightarrow 0$, $S_2 \rightarrow 1$. L'impulso su S_2 abilita ora la scrittura del bit successivo Q_1 .
- Il risultato del confronto precedente (esito “SI”, $V_a > V_{DAC}$) viene usato dalla logica combinatoria per **confermare** $Q_2 = 1$ definitivamente.
- Contemporaneamente si prova il nuovo bit ponendo $Q_1 = 1$: il codice di prova diventa $Q_2 Q_1 Q_0 = 110$.

Il DAC aggiorna quindi la propria uscita al nuovo codice:

$$V_{DAC} = 4 \text{ V} + 2 \text{ V} = 6 \text{ V.}$$

Si esegue un nuovo confronto:

$$V_a = 5.5 \text{ V} < V_{DAC} = 6 \text{ V} \implies \text{esito negativo.}$$

Questa volta il bit appena provato (Q_1) risulterà, al fronte di clock successivo, **resettato a 0** (il DAC ha superato V_a , quindi quel bit non può essere 1 nel codice finale): il codice torna a 100 per quanto riguarda $Q_2 Q_1$, e si passerà a testare l'ultimo bit Q_0 .

Algoritmo generale (n bit)

Il procedimento si generalizza a un ADC-SAR a n bit come segue: per ogni bit, dal MSB al LSB,

1. il Ring Counter abilita la scrittura del bit corrente;
2. la logica pone a 1 (in via di prova) il bit corrente, lasciando invariati i bit già decisi in precedenza;
3. il DAC converte il nuovo codice di prova in V_{DAC} ;
4. il comparatore confronta V_a con V_{DAC} ;
5. al fronte di clock successivo, in base al risultato del confronto:
 - se $V_a > V_{DAC}$: il bit resta a 1 (viene confermato);
 - se $V_a < V_{DAC}$: il bit viene riportato a 0.

Il processo richiede quindi n colpi di clock per determinare i bit, più un colpo aggiuntivo di inizializzazione/lettura: per questo il Ring Counter dell'esempio ha 5 stadi ($S_0, \dots, S_4$) per un ADC a 3 bit (1 stadio di start + 3 stadi di conversione + 1 stadio di fine conversione/lettura del risultato).

Tabella riassuntiva dell'esempio ($V_a = 5.5$ V)

Fronte di clock	Bit testato	Codice di prova $Q_2Q_1Q_0$	V_{DAC}	Confronto $V_a \geq V_{DAC}$
1 (init.)	Q_2	100	4 V	$5.5 > 4 \Rightarrow$ mantieni
2	Q_1	110	6 V	$5.5 < 6 \Rightarrow$ resetta
3	Q_0	101	5 V	$5.5 > 5 \Rightarrow$ mantieni

Il codice digitale finale è dunque $Q_2Q_1Q_0 = 101$ (decimale 5), corrispondente a $V_{DAC} = 5$ V: il valore più vicino per difetto a $V_a = 5.5$ V ottenibile con una risoluzione di 1 V (errore di quantizzazione = 0.5 V, cioè metà del passo, come atteso).

Fase 5 – Valutazione del confronto (subito dopo il secondo fronte)

Nello stesso ciclo di clock in cui Q_1 è stato posto a 1 (Fase 4), il comparatore valuta il nuovo confronto tra V_a e la V_{DAC} appena aggiornata:

$$V_a = 5.5 \text{ V} \quad \text{vs} \quad V_{DAC} = 6 \text{ V}.$$

Poiché

$$V_a = 5.5 \text{ V} < V_{DAC} = 6 \text{ V},$$

l'esito del confronto è **negativo**: il bit Q_1 appena provato *non* sarà confermato. Questo risultato viene memorizzato dalla logica combinatoria e sarà applicato al registro dati al fronte di clock *successivo* (Fase 6): la fase di “confronto” è quindi concettualmente distinta dalla fase di “scrittura”, anche se entrambe avvengono nello stesso periodo di clock.

Fase 6 – Subito dopo il terzo fronte di clock

Al terzo fronte di clock accadono, come nei casi precedenti, due cose:

- Il Ring Counter avanza: $S_2 \rightarrow 0$, $S_3 \rightarrow 1$. L'impulso su S_3 abilita ora la scrittura del bit meno significativo Q_0 .
- In base al risultato negativo della Fase 5, il bit Q_1 viene **resettato a 0**: il codice torna ad avere $Q_2Q_1 = 10$.
- Contemporaneamente si prova il nuovo bit ponendo $Q_0 = 1$: il codice di prova diventa $Q_2Q_1Q_0 = 101$.

Il DAC aggiorna quindi la propria uscita:

$$V_{DAC} = 4 \text{ V} + 0 \text{ V} + 1 \text{ V} = 5 \text{ V}.$$

Fase 7 – Confronto sull'ultimo bit (LSB)

Si esegue l'ultimo confronto della conversione:

$$V_a = 5.5 \text{ V} > V_{DAC} = 5 \text{ V} \implies \text{esito } \mathbf{positivo} \text{ (SI)}.$$

Il bit Q_0 appena provato viene quindi **mantenuto a 1**. A questo punto tutti e tre i bit sono stati testati e il codice binario è stato completamente determinato:

$$Q_2Q_1Q_0 = 101 \quad (= 5 \text{ in decimale}), \quad V_{DAC} = 5 \text{ V}.$$

Fase 8 – Subito dopo il quarto fronte di clock: fine conversione

Al quarto fronte di clock il Ring Counter avanza all'ultimo stadio ($S_3 \rightarrow 0$, $S_4 \rightarrow 1$). Questo impulso non serve più a testare un bit (tutti e 3 i bit sono già stati decisi), ma segnala la **fine della conversione**:

- Vengono attivate le porte di uscita G_A, G_B, G_C , che trasferiscono il contenuto del registro $Q_2Q_1Q_0 = 101$ alle uscite digitali finali del convertitore D_A, D_B, D_C (mostrate nello schema come $-1, -0, -1$, cioè $1, 0, 1$).
- Il risultato $101_2 = 5$ rappresenta un **arrotondamento per difetto** di $V_a = 5.5 \text{ V}$ rispetto alla risoluzione del DAC ($\Delta V = 1 \text{ V}$): l'errore di quantizzazione vale $5.5 - 5 = 0.5 \text{ V}$, pari a metà passo, come atteso per un ADC con questa risoluzione.

Il valore digitale $N_{ADC} = 101$ resta stabile in uscita (latched) finché non verrà completata una nuova conversione: “ N_{ADC} rimane costante finché il prossimo valore verrà trasferito”.

Fase 9 – Avvio di una nuova conversione

Quando si presenta un nuovo valore analogico da convertire, ad esempio

$$V_a = 3.5 \text{ V},$$

al quinto fronte di clock (che coincide, per il Ring Counter, con un nuovo primo fronte del ciclo successivo) accade quanto segue:

- Viene inviato nuovamente il segnale di **CLEAR** ai flip-flop del registro dati Q_0, Q_1 (e implicitamente Q_2): il registro si azzerava per iniziare una nuova ricerca dicotomica.
- Il Ring Counter viene resettato: $S_4 \rightarrow 0$, $S_0 \rightarrow 1$ di nuovo, esattamente come nella Fase 1 iniziale.
- Si ricomincia quindi il ciclo di conversione da capo, testando nuovamente il MSB Q_2 : il registro di prova riparte da $Q_2Q_1Q_0 = 100$ e $V_{DAC} = 4 \text{ V}$.

Le uscite finali $D_AD_BD_C$ (risultato della *conversione precedente*, 101) restano invariate in questa fase, poiché vengono aggiornate solo al termine della *nuova* conversione (di nuovo dopo 4 fronti di clock): questo è il motivo per cui il registro di uscita e il registro interno di lavoro sono tenuti separati nello schema.

Tabella riassuntiva completa (aggiornata)

Fronte	Ring Counter attivo	Bit testato	Codice $Q_2Q_1Q_0$	V_{DAC}	Esito confronto
–	S_0	(init.)	000	0 V	–
1	S_1	Q_2	100	4 V	$5.5 > 4$: mantieni
2	S_2	Q_1	110	6 V	$5.5 < 6$: resetta
3	S_3	Q_0	101	5 V	$5.5 > 5$: mantieni
4	S_4	–	101	5 V	fine conversione, latch uscite

Risultato finale: $D_AD_BD_C = 101_2 = 5$, con $V_{DAC} = 5 \text{ V} \approx V_a = 5.5 \text{ V}$ (arrotondamento per difetto).